
BACHELOR RESEARCH REPORT

DeckForge AI

*Self-trained and Deployed AI Image Generator for
Skateboard Decals*

AUTHOR

Kylian Algoet

PROGRAMME

Multimedia & Creative Technologies · Erasmushogeschool Brussel

ASSIGNMENT

Final bachelor resit assignment

REPOSITORY

<https://github.com/KylianAlgoet/Selftrained-and-deployed-AI-image-generator>

BUILT FROM COMMIT

c6afd42 · 2026-08-16

Document control

Report	DeckForge AI — Self-trained and Deployed AI Image Generator for Skateboard Decals
Author	Kylian Algoet
Programme	Multimedia & Creative Technologies, Erasmushogeschool Brussel
Assignment	Final bachelor resit assignment
Repository	<code>github.com/KylianAlgoet/Selftrained-and-deployed-AI-image-generator</code>
Public planning	<code>github.com/users/KylianAlgoet/projects/1</code>
Submission deadline	2026-08-17 06:00 Europe/Brussels

How to read this report

Every quantitative claim is traceable to a file in the repository, and the path is given where the claim is made. Experiment identifiers (`EXP-###`), decision records (`DR-###`), research questions (`RQ##`) and risks (`R##`) all resolve to real records, and a validation script asserts that they do before this PDF is built.

Numbers appearing in more than one place — test counts, the generation total, memory figures, checkpoint hashes — are not typed into the text. They are substituted at build time from `report/facts.yaml`, where each is proved against the evidence file that states it, so a figure that drifts from its evidence fails the build rather than reaching this page.

Failed and blocked work is reported as a result, not omitted. Sections 12 and 18 exist for that purpose, and the conclusions in section 19 are qualified where the evidence is qualified.

Use of AI assistance. This project was carried out with Claude Code (Anthropic) as an engineering and documentation assistant, and **visual evaluation was AI-assisted**: ChatGPT contributed visual analysis and proposed scoring at the review gates, while Kylian Algoet reviewed and approved the recorded scores and retained final authority over every production selection and research conclusion. **Every GPU generation was explicitly authorised by Kylian Algoet; no AI assistant had authority to initiate GPU inference without that approval.** No automated indicator populated a rubric cell, and **no AI assistant had final validation or decision authority over its own work.** The full account is in section 6.4, and the session-by-session record in `docs/ai-usage.md`.

Reproducing this document. `python scripts/build_report.py` renders the Markdown in `report/sources/` through the template in `report/templates/` and prints it to PDF with headless Chrome. The design, the alternatives considered and the stated limitations are recorded in `DR-015`.

Contents

1	Executive summary	
	What was built	
	What the research established	
	What did not work, and is reported as such	
	What it means	
2	Context and assignment	
	2.1 The brief	
	2.2 Mandatory requirements	
	2.3 Scope, and what was deliberately excluded	
	2.4 The constraint that shaped the project	
	2.5 Deadlines	
3	Learning outcomes	
	3.1 How the evidence was produced	
	3.2 What the outcomes are not claimed on	
4	Problem statement	
	4.1 The practical problem	
	4.2 The research problem	
	4.3 Why the constraint is interesting rather than merely inconvenient	
	4.4 What success requires	
5	Research questions	
	5.1 Primary question	
	5.2 Subquestions and where they stand	
	5.3 How the questions were kept honest	
6	Methodology	
	6.1 The decision loop	
	6.2 Measurement protocol	
	6.3 Evaluation	
	6.4 Use of AI assistance	

7	Planning	
	7.1 The original plan	
	7.2 Why the under-runs happened	
	7.3 The change log	
	7.4 The feature freeze	
8	Architecture research	
	8.1 Repository structure	
	8.2 Backend framework	
	8.3 Frontend and 3D stack	
	8.4 ML toolchain	
	8.5 Deck geometry	
	8.6 Service architecture	
9	Model and fine-tuning comparison	
	9.1 Base model	
	9.2 Fine-tuning method	
	9.3 The combined stack, and the ceiling it set	
10	Dataset methodology and licences	
	10.1 What was built	
	10.2 Licensing policy	
	10.3 The pipeline	
	10.4 Two findings that were recorded before they were convenient	
	10.5 The mitigation that was chosen, and the one that was not	
	10.6 A correction to how these findings were first described	
11	Prototypes	
	11.0 Prototype 0 — the 3D viewer	
	11.1 Prototype 1 — base-model benchmark	
	11.2 Prototype 2 — text and reference conditioning	
	11.3 Prototype 3 — LoRA smoke test	
	11.4 Prototype 4 — style learning	
	11.5 Prototype 5 — the integrated MVP	

	11.6 What the ladder produced
12	Failed and blocked experiments
	12.1 Blocked: three sources became unavailable mid-project
	12.2 The measurement that was contaminated, and the wrong explanation for it
	12.3 Defects found in the project's own tooling
	12.4 The reproducibility defect: training is not bit-reproducible
	12.5 Five defects that only a clean clone could find
	12.6 The continuous-integration failure, and what it cost to fix
	12.7 The lesson these share
13	Experiment results
	13.1 The registry
	13.2 Memory, measured across five milestones
	13.3 Training results
	13.4 Conditioning results
	13.5 Evaluation and memorisation
	13.6 Service results
	13.7 The result that closes the loop
14	The integrated MVP
	14.1 What it does
	14.2 One process, one worker — a measurement, not a preference
	14.3 Progress reporting that refuses to invent a number
	14.4 The geometry decision
	14.5 A feature added at the review gate
	14.6 Reproducibility surfaced to the user
	14.7 What the MVP does not do
15	Testing
	15.1 The suites
	15.2 What the suites deliberately do not prove
	15.3 How the layers were designed to catch different things
	15.4 Tests that encode research decisions

	15.5 Upload security
	15.6 Continuous integration
	15.7 Growth, and one restraint
16	Deployment and reproducibility
	16.1 The decision
	16.2 The clean-clone test
	16.3 The weights problem
	16.4 Demonstration readiness
	16.5 What "reproducible" means here, precisely
17	Ethics
	17.1 Training data and copyright
	17.2 Reproducing the reference image
	17.3 Privacy and untrusted input
	17.4 Bias
	17.5 Honest representation of what the system is
	17.6 What was not addressed
18	Limitations
	18.1 Product limitations
	18.2 Research limitations
	18.3 Deployment limitations
	18.4 Validity threats
	18.5 What follows from these
19	Conclusions
	19.1 The primary question
	19.2 Conclusions per research question
	19.3 The findings worth carrying beyond this project
	19.4 On the process
	19.5 What this project does not conclude
20	Reflection
	20.1 What changed from the original plan

20.2	What failed	
20.3	Decisions that saved the project time	
20.4	Evidence that changed a decision	
20.5	Lessons that were expensive	
20.6	On working with an AI assistant	
20.7	What would be done differently	

21	Lessons learned	
21.1	Technical	
21.2	Methodological	
21.3	Reproducibility	
21.4	Testing and CI	
21.5	Working with AI assistance	

22	What should be done differently	
22.1	Seed every source of randomness on the first day	
22.2	Run the clean-clone test at the first milestone	
22.3	Design the image-count experiment to answer the question asked	
22.4	Measure at least one alternative fine-tuning method	
22.5	Write the report in parallel, as planned	
22.6	Establish the deck geometry from the real target immediately	
22.7	What would not be changed	

23	Future work	
23.1	Close the reproducibility defect	
23.2	Exercise the external backup	
23.3	Measure a second fine-tuning method	
23.4	Answer the image-count question properly	
23.5	Test the mitigation that was never tested	
23.6	Scale beyond one generation at a time	
23.7	Address the deployment questions this project did not	
23.8	Product work that is genuinely optional	
23.9	The one thing that should happen first	

24	References	
	24.1 What is cited, and what is not	
	24.2 Models and methods	
	24.3 Model weights actually used	
	24.4 Dataset sources and licence regimes	
	24.5 Libraries and tools	
25	Appendices	
	Appendix A — Experiment index	
	Appendix B — Decision records	
	Appendix C — Research-question matrix	
	Appendix D — Risk register summary	
	Appendix E — Fact locks	
	Appendix F — Reproducing this document	
26	D1-D7 traceability	
	26.1 Learning outcomes	
	26.2 Mandatory requirements	
	26.3 Where each research question is answered	
	26.4 What this matrix deliberately does not do	

1 Executive summary

DeckForge AI is a working system that lets a customer of a skateboard manufacturer describe a deck graphic in words, optionally upload a reference image, choose one of three visual styles, and see the generated artwork on an interactive 3D skateboard deck before downloading it. The image model is fine-tuned locally, on a single consumer laptop GPU, from a dataset built for this project.

The engineering result is that it works. The research result is more specific, and more useful: it is a measured account of what fits in **8 GB of video memory**, what does not, and how the difference was established rather than assumed.

What was built

A local diffusion pipeline — Stable Diffusion 1.5, three per-style LoRA adapters trained for this project, and IP-Adapter for reference-image conditioning — served by a FastAPI process behind a React and Three.js frontend. Generation is direct to the deck format, 512×1536 pixels, and the result is mapped onto a procedurally generated 3D deck with correct nose-to-tail orientation.

The dataset holds **148 items** across three styles, every one of them public domain, CCo or created for this project, with its licence and source URL recorded per item before it was allowed into a training split.

What the research established

Six prototypes were built in sequence, each answering a question the next one depended on. Forty experiments are registered with their configuration, measured hardware readings and conclusions.

The central constraint governed every decision. SDXL produces better artwork than SD 1.5 at its native resolution — the project's own rubric scores say so — and it was **rejected anyway**, because at 1024×1024 it allocated 10 738 MiB on a card with 8187.5 MiB. Windows spilled silently into host memory rather than raising an out-of-memory error, so all thirty runs "succeeded". That experiment set the method for everything after it: **read every memory figure against the ceiling, never against whether the run crashed.**

The production stack fits that ceiling with **200 MiB to spare** under real serving conditions — 2.4 % of the device. This is not comfortable headroom, and the report does not describe it as such. It is why the service runs exactly one worker with one resident pipeline.

Reference conditioning went to IP-Adapter at scale 0.55 rather than img2img, on evidence that was sharper than expected: every near-copy flag in that milestone came from img2img at the deck format, some of them perceptually indistinguishable from the reference image. A method that reproduces its input is not a generator.

Style learning produced three adapters at a default weight of 0.7. Two of the three ship at **training step 300, not the 600 they were trained to** — prompt adherence fell while style consistency held, so training longer made the model more stylish and less obedient. That finding exists only because

checkpoints were saved at four points and a human compared them.

What did not work, and is reported as such

- **One style ships as a partial pass.** `retro-poster` learned the frames and display typography of the archive posters it was trained on. It is shipped with the limitation stated and a warning on every request, rather than dropped or quietly upgraded.
- **The image-count question is inconclusive.** Training on 12, 24 and 44 images produced a non-monotonic ordering, and no minimum count was established. It is reported as inconclusive.
- **Training is not bit-reproducible from its recorded seed.** The adapters must be preserved as artifacts, verified by SHA-256, because they cannot be regenerated. Inference, separately, *is* deterministic: a clean clone reproduced an earlier output byte for byte, three days later, in a freshly built environment.
- **One base-model candidate could never be measured.** SD 2.1 was gated behind authentication, so the model decision rests on two candidates rather than three.

What it means

The implemented system meets the mandatory technical requirements addressed by this report; remaining submission deliverables are tracked separately in the project planning. Where the evidence stops, the report says so. The strongest claim it makes is not that the pipeline works, but that the reasons for each choice are measured, recorded, and in several cases the opposite of what was expected at the start.

2 Context and assignment

2.1 The brief

The client is a skateboard manufacturer. The system must let a customer enter a text prompt, upload a reference image, select a visual style, generate new decal artwork with a **self-trained (locally fine-tuned) model**, view the result on an interactive 3D skateboard deck, and download the artwork.

This is a final bachelor resit assignment for Multimedia & Creative Technologies at Erasmushogeschool Brussel. The working title of the system is **DeckForge AI**.

The assignment is unusual in one respect that shaped everything: **the assessed deliverable is the research process, not only the artefact**. A working generator built without a visible, iterative, evidence-based process would not satisfy it. That is why this report spends as much space on measurements that changed decisions, and on approaches that did not work, as on the system that resulted.

2.2 Mandatory requirements

Thirteen requirements are stated in the assignment. Section 26 traces each to its evidence; this table states them, and where in this report each is addressed.

#	requirement	addressed in
1	Collect and create a custom training dataset	§10
2	Document dataset provenance and permitted usage	§10, §17
3	Support multiple visual styles (≥ 3 visually distinct)	§11.4, §13
4	Train or fine-tune the model locally	§9.2, §11.3, §11.4
5	Combine text prompting and a reference image	§9.3, §11.2
6	Generate new decal artwork	§14
7	Map it onto a 3D skateboard	§11.0, §14
8	Provide a reproducible deployment or demonstration setup	§16
9	Maintain a public planning link	§7
10	Provide research documentation as PDF	this document
11	Provide prototype evidence	§11, §13
12	Provide the final GitHub result	§16

#	requirement	addressed in
13	Provide a presentation as PDF	not addressed by this report — see §26 and DR-016

Requirement 13 is not discharged by this report. It belongs to the presentation milestone (M10), which had not run when this report was first written; the deck has since been built and is recorded in §26 and DR-016/DR-017, but it **has not passed a human visual gate** and **no rehearsal has been run**. Claiming it complete here would be exactly the kind of unevidenced statement the rest of this document avoids.

2.3 Scope, and what was deliberately excluded

The MVP covers: prompt and optional negative prompt, an optional PNG/JPG/WEBP reference image, style selection, reference strength, seed, generation, an interactive 3D deck preview with correct nose-to-tail orientation, and download.

Explicit non-goals, fixed at the start of the project and never revisited: user accounts, payments, social features, a webshop, native applications, and production-scale infrastructure. Any addition to scope required an entry in the planning change log, which is how the two features that *were* added late — real generation progress and local decal upload — are traceable rather than silently absorbed (§7.3).

2.4 The constraint that shaped the project

The system had to be built and trained on the machine available, audited before any decision was taken:

GPU	NVIDIA GeForce RTX 4060 Laptop, 8187.5 MiB VRAM
System memory	16 GB
OS / shell	Windows 11 Home, PowerShell 5.1
Python	3.11 (3.14 is the system default; 3.11 was used for all PyTorch [17] work)
Node	24.18.0
Not available	FFmpeg, nvcc, conda

Full audit: `docs/technical/environment-audit.md`.

Eight gigabytes of video memory is the single fact that explains most of this report. It ruled out the model that produced the best artwork (§9.1), forced the service to run exactly one worker (§14), set the ceiling that every later addition had to fit inside (§9.3), and turned "does it fit" into a question that had to be measured before each step rather than assumed after it.

The environment audit itself was later found to contain an error — the recorded Node version had gone stale — and that is reported in §12 rather than quietly corrected, because a project whose audit drifts is a project whose claims drift.

2.5 Deadlines

event	date
Feature freeze	2026-08-15, brought forward to 2026-08-09
Final content	2026-08-16 18:00
Submission	2026-08-17 06:00 Europe/Brussels
Presentation	2026-09-02

The freeze was brought forward six days because the implementation milestones finished early. The reasoning, and what the freeze does and does not permit, is in §7.4.

3 Learning outcomes

Seven learning outcomes are assessed. They are stated here with the evidence that carries each one; the complete mapping, including file paths and experiment identifiers, is in §26 and in `docs/learning-outcome-traceability.md`, which was appended to at every milestone rather than assembled at the end.

ID	outcome	principal evidence in this report
D1	Independent applied research	§5 research questions · §13 forty registered experiments · §12 results that refuted their own hypotheses
D2	Independent professional functioning	§7 planning and gates · §12 blocked sources escalated rather than worked around · §17 licence policy enforced before collection
D3	Iterative planning and professional methodology	§7.3 thirteen planning change-log entries with reasons · §6.3 the two-gate review protocol
D4	Comparison and application of multiple solution methods	§8 weighted architecture matrices · §9 base-model and fine-tuning comparisons · §11.2 four conditioning arms
D5	Complex problem solving via prototypes and new technologies	§11 six prototypes, each answering the question the next depended on · §12 the defects found along the way
D6	Justified research conclusions	§13 conclusions tied to registry rows · §18 limitations that qualify them · §19
D7	Professional documentation and presentation	this report · §16 the runbook and weights manifest · the evidence set under <code>docs/evidence/</code>

3.1 How the evidence was produced

Two habits account for most of the evidence behind these outcomes, and both were adopted early enough to shape the work rather than describe it.

Thresholds were written down before results were read. Noise floors, pass conditions and tolerances live in code and in plans that predate the runs they judge. This is what allows §9.2 to report a diagnostic as a diagnostic rather than promoting it to a pass after the fact, and what makes "the adapter changed the output" a measured claim instead of an impression.

Human judgement was separated from automated measurement, and the separation was enforced. Visual evaluation was **AI-assisted** — ChatGPT contributed visual analysis and proposed scoring at the review gates — and **Kylian Algoet reviewed and approved every recorded score and retained final decision authority** (§6.4). Separately, the offline indicators — perceptual hashes, CLIP similarity — populate **no** rubric cell, select no checkpoint and decide no verdict; they live in separate files from human judgement, and §6.3 explains why that boundary was drawn where it was. At the first review gate the completed score sheet was hashed **before** the blinding map was opened, so "no score was edited after unblinding" is a check rather than a promise, and a test still asserts it.

3.2 What the outcomes are not claimed on

D4 asks for comparison of multiple solution methods. Four of the five mandated fine-tuning methods were **screened on criteria and never executed** (§9.2). The comparison that exists is real and documented; it is not a measured five-way benchmark, and this report does not present it as one.

D6 asks for justified conclusions. **Four of the twelve research questions are only partially or boundedly answered — RQ1, RQ4, RQ7 and RQ11 — and RQ4's image-count component is explicitly inconclusive** (§5.2). They are reported that way. A learning outcome is not better served by a tidy answer than by an honest one.

4 Problem statement

4.1 The practical problem

A skateboard manufacturer wants customers to design their own deck graphics. The obstacle is not demand but production: commissioning artwork per customer does not scale, stock template libraries produce decks that look like everyone else's, and a generic image generator produces images that are not decal artwork — wrong proportions, wrong composition, and no way to see the result on a board before ordering.

Three properties are therefore required together, and it is the combination that is hard:

1. **The output must be a decal**, not a picture of one. A skateboard deck is roughly 1:3.6; almost every diffusion model is trained on square or near-square images and drifts toward photographic product mockups when asked for a deck (§9.1, Figure 1).
2. **The style must be the manufacturer's**, consistently, across arbitrary prompts. A pretrained model has no notion of any particular house style, so a style has to be taught.
3. **The customer must be able to steer it**, both with words and with an image they already have, without the result becoming a copy of that image (§11.2).

4.2 The research problem

The assignment adds a constraint that turns an engineering task into a research one: the model must be **trained or fine-tuned locally**, and the only machine available has **8187.5 MiB of video memory**.

That constraint is not a detail of implementation. It determines which base model can be used, which fine-tuning method is even possible, whether reference conditioning and a trained adapter can be resident at the same time, whether the deck's tall geometry can be generated directly or must be reconstructed from something squarer, and how many processes the finished service may run.

None of those questions has a reliable answer in advance for a specific 8 GB card, because the figures that circulate for these models are typically quoted for larger GPUs or with optimisations whose costs are not stated. They had to be measured on this machine.

The problem this project addresses is therefore:

How can a locally fine-tuned diffusion model, conditioned on both a text prompt and a reference image, generate skateboard-decal artwork in multiple visually distinct styles with reproducible quality on consumer hardware with 8 GB of VRAM?

4.3 Why the constraint is interesting rather than merely inconvenient

An 8 GB budget makes the trade-offs visible that a larger card would hide. Three of this project's more useful findings exist only because the ceiling was close enough to hit:

- A model can report thirty successful runs while allocating more memory than the card physically holds, because Windows spills into host memory instead of failing (§9.1). On a 24 GB card that experiment returns "fine" and teaches nothing.
- The marginal cost of a trained adapter (+3.04 MiB) and of reference conditioning (~1 249 MiB) are worth measuring separately only when the sum is close to the limit — and it is their sum, not either one, that sets the production ceiling (§9.3).
- A service that holds one pipeline resident and refuses a second worker is an architectural consequence of a memory measurement, not a preference (§14.2).

4.4 What success requires

The system is successful if a customer can go from a sentence to a decal on a rotating 3D deck, in one of at least three recognisably different styles, on this hardware, reproducibly — and if the reasons for each design decision are traceable to evidence rather than to taste.

The second half of that sentence is the part the assignment assesses, and it is why the twelve research questions in §5 exist.

5 Research questions

5.1 Primary question

How can a locally fine-tuned diffusion model, conditioned on both a text prompt and a reference image, generate skateboard-decal artwork in multiple visually distinct styles with reproducible quality on consumer hardware (8 GB VRAM)?

A working hypothesis was recorded at the same time, and deliberately labelled a hypothesis rather than a plan:

A LoRA fine-tune of a pretrained SD 1.5-class diffusion model, trained locally per style (or as one multi-style LoRA), combined with an image-conditioning method (img2img, IP-Adapter, or ControlNet), will produce acceptable multi-style decal artwork within the hardware and time budget.

It survived, but not untouched. Three of its clauses were settled against expectation: the multi-style option was built and rejected on evidence (§11.4), img2img was rejected as the primary conditioning method for a reason nobody anticipated (§11.2), and "reproducible" turned out to mean two different things for inference and for training (§18.2).

5.2 Subquestions and where they stand

Twelve subquestions were registered before any experiment ran, each with a hypothesis, a method and the prototype expected to answer it.

Eight are answered within their stated scope: RQ2, RQ3, RQ5, RQ6, RQ8, RQ9, RQ10 and RQ12. Four are only partially or boundedly answered: RQ1, RQ4, RQ7 and RQ11. RQ4's image-count component remains explicitly **inconclusive**.

RQ1 is counted as bounded rather than answered for a specific reason: it asks which method is *feasible and most effective*. Feasibility is demonstrated; **"most effective" was never established, because four of the five candidate methods were screened and never measured** (§9.2).

RQ	question	status	answered by
RQ1	Which fine-tuning method is feasible and most effective on 8 GB?	feasibility only	§9.2 · EXP-016...019 · DR-009
RQ2	Which pretrained base model is feasible on this hardware?	answered, two candidates	§9.1 · EXP-002, EXP-004 · DR-007
RQ3	How can a legally usable custom dataset be created?	answered	§10 · DR-006

RQ	question	status	answered by
RQ4	How many images per style, and what caption standards?	INCONCLUSIVE on count	§11.4 · EXP-024n12/n24
RQ5	One multi-style LoRA or separate style LoRAs?	answered	§11.4 · EXP-030 · DR-010
RQ6	How should text and reference conditioning be combined?	answered	§11.2 · EXP-008...014 · DR-008
RQ7	Which generation parameters most influence quality?	partially answered	§11.2, §11.4 · EXP-008/009/031
RQ8	How should the deck aspect ratio be handled?	answered, hypothesis refuted	§9.1 · EXP-005 · DR-007
RQ9	How is the decal mapped onto a 3D deck?	answered	§11.0 · DR-005, DR-012
RQ10	How should decals be evaluated objectively?	answered, with a stated threat	§6.2
RQ11	What are the copyright, privacy, bias and ethics constraints?	partially answered	§17
RQ12	What deployment setup is reproducible on this hardware?	answered	§16 · DR-014

The four that are bounded

RQ1 is bounded to feasibility. See above: the "most effective" half of the question is unanswered, and DR-009 says so rather than implying a comparison that did not happen.

RQ4's image-count half is inconclusive. Training the lead style on 12, 24 and 44 images at equal compute produced a **non-monotonic** ordering, and **no minimum image count was established**. Two further limits apply and are not softened: the arms were matched on *compute*, not on epochs, so each smaller set saw its images far more often (25.0, 12.5 and 6.8 presentations per item respectively); and the comparison ran on *minimal-geometric* only, so it does not generalise to the other two styles. The caption half of RQ4 *was* answered, by a blinded A/B (§11.4).

RQ7 is partially answered. Reference strength and conditioning scale were swept properly, and LoRA weight was compared across a matrix. **Rank and learning rate were set, not swept** — rank 8 and 1e-4 were fixed at the smoke-test stage and never varied, so no claim is made about their optimality.

RQ11 is partially answered. Licensing, provenance and privacy are settled and enforced. The memorisation question is not: 0 of 252 outputs were flagged as near-copies of training images, and a perceptual-hash threshold is a **coarse indicator, not proof** that a model has not memorised anything.

5.3 How the questions were kept honest

Two protocol rules did most of the work.

A hypothesis that cannot be refuted was rewritten before it was tested. One style-learning hypothesis originally read that a trained adapter would be "at least as strong" as a baseline — a claim equality satisfies, so no result could refute it. It was replaced with four explicit verdict rules before Phase B ran.

Results that refuted their own hypothesis were kept. RQ8 predicted that direct tall generation would degrade composition and that generate-then-crop would be more reliable. The measurements said the opposite, and the recorded conclusion says so (§9.1). The same happened to a timing anomaly initially blamed on thermal throttling, which was tested and ruled out (§12.2).

6 Methodology

6.1 The decision loop

Every significant decision in this project followed the same thirteen steps, recorded in `CLAUDE.md` before the first line of code was written:

```
problem -> research question -> alternatives -> criteria -> experiment -> execution  
-> actual results -> comparison -> justification -> implementation -> validation  
-> documentation -> atomic commits
```

The rule that gives it force is negative: **do not build the complete result first and invent the process afterwards**. Fourteen decision records exist because fourteen decisions went through this loop; each states its context, the alternatives, the criteria, the decision, and — the part most often missing from such records — what it does **not** claim.

6.2 Measurement protocol

One configuration per fresh operating-system process. Adopted after an early experiment produced a 20× timing spread for provably identical work: four geometries had shared one process, and the CUDA caching allocator's state carried across them. The obvious explanation, thermal throttling, was tested and **ruled out** — a hotter, more throttled card ran the same work faster. Every VRAM and latency figure in this report comes from a process that ran one configuration.

Comparisons vary one factor against a frozen kit. Fixed prompts, fixed reference images, fixed seeds (42, 1337, 2026), fixed resolution and sampler settings. The prompt kit is hash-locked (`c40749bc...`) and asserted unchanged by a test, so a comparison cannot be quietly invalidated by an edited prompt.

Memory is quoted against the ceiling, never against whether the run crashed. This convention comes directly from the SDXL result in §9.1, where thirty runs "succeeded" while allocating more than the card holds.

Peaks are separated by phase. Post-load, forward/backward and optimizer-step peaks are recorded separately rather than as one process maximum. That is what made the mechanism in §9.2 readable: activations scale with geometry, optimizer state does not.

Thresholds are declared before results are read. Noise floors and tolerances live in code that predates the runs it judges.

6.3 Evaluation

A nine-dimension rubric, scored 1–5, was defined **before** the first experiment: prompt adherence, style consistency, reference influence, visual quality, decal suitability, composition, artefacts, originality, and diversity across seeds.

Scoring was AI-assisted, and the division of labour is recorded in the gate artifacts

themselves. ChatGPT contributed visual analysis and proposed scoring of the contact sheets; Kylian Algoet reviewed the material, approved the recorded scores, and made every production selection and research conclusion. The Gate-1 scoring file's own header records its reviewer line as "ChatGPT visual review with Kylian present", and the Gate-2 file records "Final human approver: Kylian Algoet" with "Visual-analysis assistance: ChatGPT". Both are preserved unedited.

Three rules govern how scores are recorded, and each exists because the alternative would have flattered the results:

- **A blank is never a zero, and is never back-filled.** Twenty-nine score cells in one milestone are recorded as *not scored* and excluded from every mean rather than imputed. Text-only generation at the deck format was left unscored rather than reusing a comparable value from an earlier milestone.
- **Automated indicators decide nothing.** Perceptual-hash and CLIP-similarity measures support the rubric and never replace it. They populate no rubric cell, select no checkpoint and set no verdict, and they are stored in separate files from human judgement.
- **Objective measurements and human scores are never blended.** Decision records report them in separate sections.

The two-gate review protocol

Style learning used **two** human review gates rather than one, because they ask different questions:

	Gate 1 — pilots	Gate 2 — production selection
question	which configuration goes forward	which checkpoint ships
sheets	blinded within style	labelled
why	the arms differed in one hidden variable each, so a label would leak the answer	the question cannot be answered without knowing which checkpoint a sheet is
known cost	none beyond reduced context	labelled sheets carry an expectation effect the blinded ones did not — stated rather than left implicit

Both gates hash the completed score file. The Gate-1 hash was supplied **before** the blinding map was opened, verified twice, and is asserted by a test, with the file pinned in `.gitattributes` so line-ending normalisation cannot alter it. "No score was edited after unblinding" is therefore checkable.

6.4 Use of AI assistance

Claude Code (Anthropic) was used throughout as an engineering and documentation assistant, under a written working agreement in `docs/ai-usage.md` that predates the work. Fifteen dated session entries record what it did, what the student decided, and where its own output was wrong. This section states the boundary; the log holds the detail.

category	what it covered
Human decisions	every decision-record conclusion · approval of all recorded rubric scores · the Gate-1 and Gate-2 selections · the texture-fit choice, quoted verbatim in DR-012 · the CI remedy · authorising every GPU generation
AI-assisted evaluation	ChatGPT contributed visual analysis and proposed scoring of the contact sheets at both review gates, with Kylian reviewing and approving the recorded result
AI-assisted planning	milestone plans — which the student's review changed : the two-phase split of the style-learning milestone, twelve mandatory corrections to the MVP plan, and the correction that gradient accumulation is not a memory tier
AI-assisted implementation	the training and inference runners, the API, the frontend, the test suites, the dataset and evaluation tooling
AI-assisted documentation	this documentation set, written from executed results
AI-assisted review and debugging	the reproducibility diagnosis behind R14, the five defects found during testing and deployment, the continuous-integration stall trace
GPU execution	all 31 generations. Every one was explicitly authorised by Kylian Algoet; no AI assistant had authority to initiate GPU inference without that approval
Produced by tools	VRAM figures, timings, hashes, perceptual and CLIP indicators, test counts

Three statements are load-bearing and are made without hedging.

No AI assistant had final validation or decision authority over its own work. Measurements come from instrumented runs; **AI-assisted visual evaluation was reviewed and approved by Kylian Algoet**, who retained final authority over every production selection and research conclusion. **No offline indicator ever populated a rubric cell or selected a checkpoint, weight, style or verdict** — that boundary is absolute and is asserted in the gate records.

It stopped rather than deciding when a decision was not its to make. When the clean-clone test failed on a value documented as frozen across an earlier milestone's evidence, it diagnosed the cause, prepared two options, and stopped — repointing that constant would have moved a fingerprint that milestone's records cite as unchanged.

Its own work produced defects, and they are recorded rather than absorbed. Four browser assertions failed on first run and were all test defects rather than application defects; a fixture reader written with a regex failed against real TypeScript and was replaced rather than patched; an experiment runner defect is preserved in the evidence as a failed row (§12.3). §12 reports these alongside the project's other failures, because a defect found in the tooling is worth the same as one found in the product.

7 Planning, original and changed

7.1 The original plan

Twelve milestones (M0–M11) were planned on 2026-07-27, the first day, covering roughly 132 focused hours across 21 days. The plan is published as a public GitHub Project mirroring each milestone as a repository issue with objective, acceptance criteria, dates, dependencies and expected evidence — satisfying the assignment's public-planning requirement.

One rule governs the planning document: the original plan is never rewritten. Adjustments are appended to a change log with their reasons. A plan that is edited to match what happened is not evidence of planning; it is evidence of hindsight. This means the v1 table still shows later milestones as *not started* even though they are complete, and that apparent staleness is the convention working as intended.

M	milestone	planned	actual	estimate → actual
M0	Phase 0 foundation	Jul 27–28	Jul 27	6 h
M1	Prototype 0 — 3D viewer	Jul 28–30	Jul 27	10 h → ~4 h
M2	Dataset research and pipeline	Jul 30 – Aug 3	Jul 27	16 h → ~5 h
M3	Prototype 1 — base-model benchmark	Aug 2–4	Jul 30	10 h → ~8 h
M4	Prototype 2 — reference conditioning	Aug 4–6	Aug 1	10 h → ~9 h
M5	Prototype 3 — LoRA smoke test	Aug 6–8	Aug 4	10 h → ~5 h
M6	Prototype 4 — style learning	Aug 8–11	Aug 5	14 h → ~10 h
M7	Prototype 5 — integrated MVP	Aug 10–13	Aug 7	18 h → ~13 h
M8	Testing, deployment, demo	Aug 13–15	Aug 9	10 h → ~11 h

Every milestone but one came in early, and the buffer compounded: M1 and M2 finishing on day one bought roughly two days, which let the benchmark start early, which let conditioning start early, and so on. **M8 is the only milestone that overran its estimate**, by about an hour, and §7.3 explains why that overrun was worth more than the savings.

7.2 Why the under-runs happened

The under-runs are recorded with measured causes rather than attributed to good luck, because two of them changed later estimates.

Infrastructure was reusable to a degree the estimates did not anticipate. The MVP milestone added no modelling of its own: it reuses the conditioning attachment with its pinned-encoder workaround, the frozen prompt kit and the adapter-scale mechanism unchanged, which is also why its

memory behaviour is directly comparable with the experiments that preceded it.

Training was far cheaper than budgeted. Three hundred steps cost 91 seconds. The "long training run" the 10-hour estimate assumed never existed, and the entire memory-escalation contingency went unused because nothing ever escalated past the lowest tier.

A procedural decision removed a whole work package. Generating the deck geometry procedurally rather than sourcing a model eliminated model licensing, UV repair and import work from the first milestone.

7.3 The change log

Thirteen entries record every adjustment with its reason and its impact. Six matter to the research narrative.

The style set changed before collection. The planned graffiti/street-art style was replaced with ukiyo-e woodblock because graffiti photography is generally artist-copyrighted while ukiyo-e has large institutional public-domain supply. A licensing risk was designed out rather than mitigated later (§17).

A style was relabelled before any experiment ran. retro-comic became retro-poster after the student's own pre-check found the label contradicted the collected material: it is silkscreen poster work with no halftone, panels or sequential art. The wrong label had already reached the draft prompt kit, and every later style comparison would have been built on it. Correcting it cost about an hour; finding it later would have invalidated the comparisons.

Two dataset sources became unavailable mid-collection and the shares were shifted to already-approved alternatives rather than by adding new sources (§12.1).

A review gate was split in two after the student's plan review. The style-learning plan originally promised a review gate and then authorised the full runs and the final matrix before anything had been seen. The split into pilots → gate → approved runs → gate is what made the first gate a decision point rather than a formality, and it produced three further corrections including the equal-compute image-count arm.

Scope was added twice, deliberately and recorded. Real generation progress and local decal upload were both scoped by the student after using the application, neither was in the plan, and both are logged with their reasons. The progress work produced a decision record and found three defects no test could see.

A generation budget was exceeded, and recorded rather than absorbed. The research cap of 25 was reached exactly. Generation 26 was the student's own review run and 27 was deployment validation; neither carries an experiment identifier and neither is in the registry, because adding them would contaminate a frozen matrix. The total is reported as **31**.

7.4 The feature freeze

The plan set the freeze at 2026-08-15. It was brought forward to **2026-08-09**, the day the implementation work actually finished, on the reasoning that a freeze starting when the work is done is worth more than one starting on a calendar date, and that the remaining risk was no longer "does it work" but "does a late change break something that was working".

The freeze permits blocking bug fixes, documentation, test fixes, demo preparation and evidence organisation. It forbids retraining, new styles, architecture changes, UI redesign, new generation features, model replacement, geometry changes and dependency upgrades without a new decision record. This report was written under it, and the only decision record it required is DR-015, for the build that produced this PDF.

The freeze also fixes what "done" means: it lists **eight accepted limitations** and states that they are limitations rather than unfinished work (§18). That distinction was made at a review gate, before this report existed, so it could not be made retroactively to suit the writing.

8 Architecture research

Six architecture decisions were taken with weighted criteria matrices before or during construction. Four were reasoned judgements made in the first days, when no measurements existed. Two were taken later **with measured hardware data**, and they are the ones §9 covers in detail.

Each criterion is weighted 1–5 by importance, each alternative scored 1–5. The weighted total justifies the reasoned decision rather than replacing it — a matrix that outputs a decision nobody can explain is a spreadsheet, not a method.

8.1 Repository structure

criterion (weight)	monorepo	multi-repo	flat folder
Jury traceability: one history tells the whole story (5)	5	2	3
Tooling simplicity on one machine (4)	4	2	5
Deadline risk (4)	5	2	4
Reproducibility / clean-clone test (4)	5	3	3
Separation of concerns (3)	4	5	1
weighted total (max 100)	93	51	69

Monorepo (DR-001). A single Git history is itself evidence for the assessed process: the commit sequence, the experiments and the documentation move together and can be read as one story.

8.2 Backend framework

criterion (weight)	FastAPI	Flask	Node/Express
Native fit with a Python ML stack (5)	5	5	1
Built-in validation for untrusted input (5)	5	2	3
Async and long-running requests (4)	5	2	4
Auto-generated API documentation (3)	5	2	3
Testability (4)	5	4	2
Time to productivity in 19 days (4)	4	4	3
weighted total (max 125)	121	81	65

FastAPI with Pydantic [18] (DR-002). Express would have forced a second process boundary between the API and the model for no benefit. The validation criterion is weighted at 5 because every upload is untrusted input reaching an image decoder (§17.3).

8.3 Frontend and 3D stack

criterion (weight)	React + Vite + TS + R3F	plain Three.js	SvelteKit + Threlte
3D scene and UI state integration (5)	5	2	4
Ecosystem for product viewers (4)	5	4	2
Type safety for the API contract (4)	5	4	4
Testing story (4)	5	3	4
Time to first prototype (4)	4	3	3
weighted total (max 105)	101	66	72

React, Vite, TypeScript and React Three Fiber [19] (DR-003), with plain Three.js recorded as the revision path. Prototype 0 was scheduled immediately afterwards precisely to test this choice against reality rather than leave it on paper (§11.0).

8.4 ML toolchain

criterion (weight)	Diffusers + PEFT	kohya-ss sd-scripts	ComfyUI
Scriptable reproducibility: configs, seeds, testable (5)	5	4	2
Integration into the API service (5)	5	3	2
Memory optimisation for 8 GB (4)	4	5	3
Evidence value: readable code, not a black box (4)	5	3	2
Community-validated LoRA quality (3)	4	5	4
weighted total (max 105)	98	83	54

Diffusers, PEFT and Accelerate (DR-004), with kohya-ss held as a fallback if memory proved too tight. Note that kohya scores *higher* on memory optimisation and was still not selected: the scriptability and evidence criteria outweighed it, and the decision recorded that the fallback would be taken if measurements demanded it.

They did not. Training fitted at the lowest memory tier at both geometries with no escalation (§9.2), so the fallback was formally retired rather than left as an open option.

8.5 Deck geometry

The deck model could have been sourced, bought, hand-modelled or generated procedurally.

Procedural generation was selected (DR-005): it removes third-party model licensing entirely, guarantees a UV layout the project controls, and made the nose-to-tail orientation question testable rather than inherited.

This decision removed a whole work package from the first milestone and is the main reason it finished in about four hours against a ten-hour estimate (§7.2). It also has a cost, discovered four milestones later: the test decals bundled with the viewer were 512×2000, which **concealed the mismatch between the generated 1:3 decal and the deck's 1:3.902 UV domain** until the MVP was built (§11.5).

8.6 Service architecture

The last architecture decision was taken with measured data and is the most constrained of the six. Its inputs are in §9.3: the production stack leaves **200 MiB spare** under real serving conditions.

option	verdict
One process, one worker, one resident pipeline	selected (DR-011)
Multiple workers	impossible — a second resident pipeline does not fit in 200 MiB
Load per request	rejected — a cold load costs ~30 s against ~12 s resident
A job queue with a separate worker process	rejected — same memory arithmetic, plus a component the deadline could not absorb

The service runs exactly one API process with one worker, holding one pipeline resident, and serving one generation at a time behind a process-local lock. A startup guard rejects a worker count above 1.

This is a consequence of a measurement, not a preference, and it is stated in the report as a limitation rather than as a design virtue (§18.3): **scaling this system is not a configuration change**. It would need a second GPU or a smaller resident footprint.

9 Model and fine-tuning comparison

Two decisions are recorded here: which pretrained base model the system generates from, and which fine-tuning method teaches it a style. Both were taken on measured data, and both carry limitations that are stated rather than smoothed over.

9.1 Base model

Three candidates were planned. One could never be measured.

candidate	outcome
Stable Diffusion 1.5	measured — selected (DR-007)
SDXL base 1.0	measured — rejected on memory, not on quality
SD 2.1 base	blocked — the repository returns HTTP 401

SD 2.1 was gated behind authentication. Two sibling repositories from the same organisation returned 401 while SDXL returned 200, so this was repository gating rather than an outage. Per the approved plan the student was asked rather than the assistant authenticating or silently substituting a mirror; three alternatives were declined and recorded, including an ungated community mirror rejected because its fidelity to the original cannot be verified while the original is gated.

The base-model decision therefore rests on two measured candidates, not three. That is a limitation of the evidence, and it is carried into section 18 rather than left implicit.

The measurement design that changed the answer

Scoring every model at 512×512 would have been unfair to SDXL, which was never designed for it. The benchmark was therefore split into two tracks, reported separately and never averaged:

- **Track A** — every candidate at 512×512, so speed, memory and reliability are comparable.
- **Track B** — every candidate at its designed resolution, SDXL at 1024×1024.

model	geometry	peak allocated	peak device	median latency	quality (student)
SD 1.5	512×512	2 675.38	4 359.5	4.069 s	4
SD 1.5	512×768	2 979.33	4 977.5	6.811 s	—
SDXL	512×512	7 859.26	8 187.5	16.512 s	4
SDXL	1024×1024	10 738.08	8 187.5	118.733 s	5

All memory figures in MiB, against a physical 8187.5 MiB. Evidence: [experiments/registry.csv](#) EXP-002, EXP-004; [docs/evidence/prototype-1/](#) .



Figure 1. Track B — each candidate at its designed resolution, identical prompts and seed 42. SDXL at 1024×1024 produces flat artwork with no mockup framing; SD 1.5 at 512 px tends toward photographic product mockups. The literal-deck reading is resolution-dependent, not a property of all models.

[docs/evidence/prototype-1/cross-model-track-B-seed42.jpg](#)

The result that set the project's method

SDXL reported **30 of 30 successful runs at 1024×1024**. It also allocated **10 738 MiB and reserved 14 510 MiB on a card holding 8187.5 MiB**.

THE FAILURE THAT DID NOT LOOK LIKE ONE

Windows WDDM spilled the overflow into shared host memory instead of raising a CUDA out-of-memory error. Resident host memory rose to 6 806 MiB, no exception was thrown, and the pipeline's memory-tier escalation never triggered. A run that reports success is not evidence that it fitted.

Every memory figure in this report is consequently quoted **against the device ceiling**, and that convention comes directly from this experiment. It is also why the 200 MiB production margin in section 14 is never called comfortable headroom.

SDXL is retained as the visual-quality benchmark. Its advantage is real and exists only at a resolution this GPU cannot hold.

9.2 Fine-tuning method

The assignment mandates comparing five methods. They were **not** all measured, and this section says so plainly.

method	treatment	why
Training from scratch	screened out	infeasible on 8 GB and a 19-day budget by orders of magnitude
Full fine-tuning	screened out	the full UNet in optimizer state does not fit

method	treatment	why
DreamBooth	screened, never run	subject-driven; the task is style, and budget allowed one method to be measured properly
Textual Inversion	screened, never run	learns an embedding, not style weights; same budget constraint
LoRA	measured	selected (DR-009)

WHAT DR-009 DOES NOT CLAIM

LoRA is **not** shown to be the best of the five. Four of them were screened on criteria and never executed. The defensible statement, and the one the decision record makes, is that LoRA is the mandated method **demonstrated feasible on this hardware**.

What was measured

A rank-8 LoRA [3], implemented with PEFT [16], on the UNet attention projections, with the text encoder and VAE frozen. Thirteen runs across eight experiments, every one at the lowest memory tier, with no escalation anywhere.

run	geometry	steps	peak allocated	spare	s/step
EXP-016a	512×512	1	3 114.09	3 920.0	1.9344
EXP-016b	512×512	10	3 133.40	3 902.0	0.4340
EXP-016	512×512	300	3 133.40	3 902.0	0.2854
EXP-017a	512×1536	1	5 160.96	1 758.0	2.5533
EXP-017b	512×1536	10	5 182.58	1 738.0	1.1223

Three findings came out of separating the memory peaks by phase rather than recording one process-level maximum:

- 1. Activations scale with geometry; optimizer state does not.** Post-load allocation is identical at both geometries (2 066.56 MiB) and the optimizer-step peak barely moves (2 108.93 → 2 118.76), while the forward/backward peak rises from 3 114 to 5 183. Gradient checkpointing is therefore the correct first escalation, and a lower-memory optimizer would have been the wrong move.
- 2. A rank-8 adapter costs +3.04 MiB, and that cost does not scale with output size.** Measured independently three times.
- 3. Training is cheap.** Three hundred steps in 91 seconds, which made the style-comparison grid in the next prototype affordable many times over.

Proving the adapter does something

An adapter that loads without error has not been shown to work. Thresholds were written down before any result was read:

```
weight 0.0 -> 4/4 byte-identical to the no-adapter baseline
               recorded as a DIAGNOSTIC, never a pass condition
weight 1.0 -> 4/4 beyond a pre-declared noise floor
               mean |pixel diff| >= 0.5 AND >= 1% of subpixels differing
               measured: 51.89-66.33 mean diff, dHash 20-28
```

The weight-0.0 arm is recorded as a diagnostic deliberately: loading an inactive adapter can legitimately alter the execution graph, so byte-identity was a welcome result that was never allowed to become a pass criterion. Equally, **a differing image hash alone was never treated as sufficient** evidence that the adapter had changed anything.

9.3 The combined stack, and the ceiling it set

The two decisions above have to hold *simultaneously*, and that was genuinely uncertain: IP-Adapter alone had already peaked at 7 965.5 MiB of 8187.5 at the deck format.

Both adapters were confirmed live at once by reading the UNet back — 128 LoRA modules and 16 IP-Adapter attention processors — rather than inferred from a call that returned without raising.

The combined stack fits at 512×1536 by 202 MiB, later measured at **200 MiB** under real serving. That is *less* margin than IP-Adapter alone had, because the adapter takes roughly 20 MiB of device memory on top. No overflow flag fired and no tier escalated — which is precisely the pattern that made SDXL look viable until its figures were read against the ceiling.

Geometry was never reduced to make this pass. The result re-scoped the risk rather than closing it: the open question stopped being *does this stack fit* and became *does anything added to it still fit*.

10 Dataset methodology and licences

10.1 What was built

148 items across three styles, every one public domain, CCo or created for this project, with its licence and source recorded per item before it was permitted into a training split.

style	items	licence	source
ukiyo-e woodblock	55	CC0 [14]	Metropolitan Museum of Art, open access [12]
minimal geometric	52	project-original	seeded programmatic generation
retro silkscreen poster	41	public domain	Library of Congress — WPA / Federal Theatre Project [13]
total	148		

Splits: **124 train, 17 validation, 7 holdout**. The holdout items were never seen by any training run and are the reference images used to test conditioning (§11.2), which is what makes "the adapter has not simply memorised its inputs" a testable statement rather than an assumption.

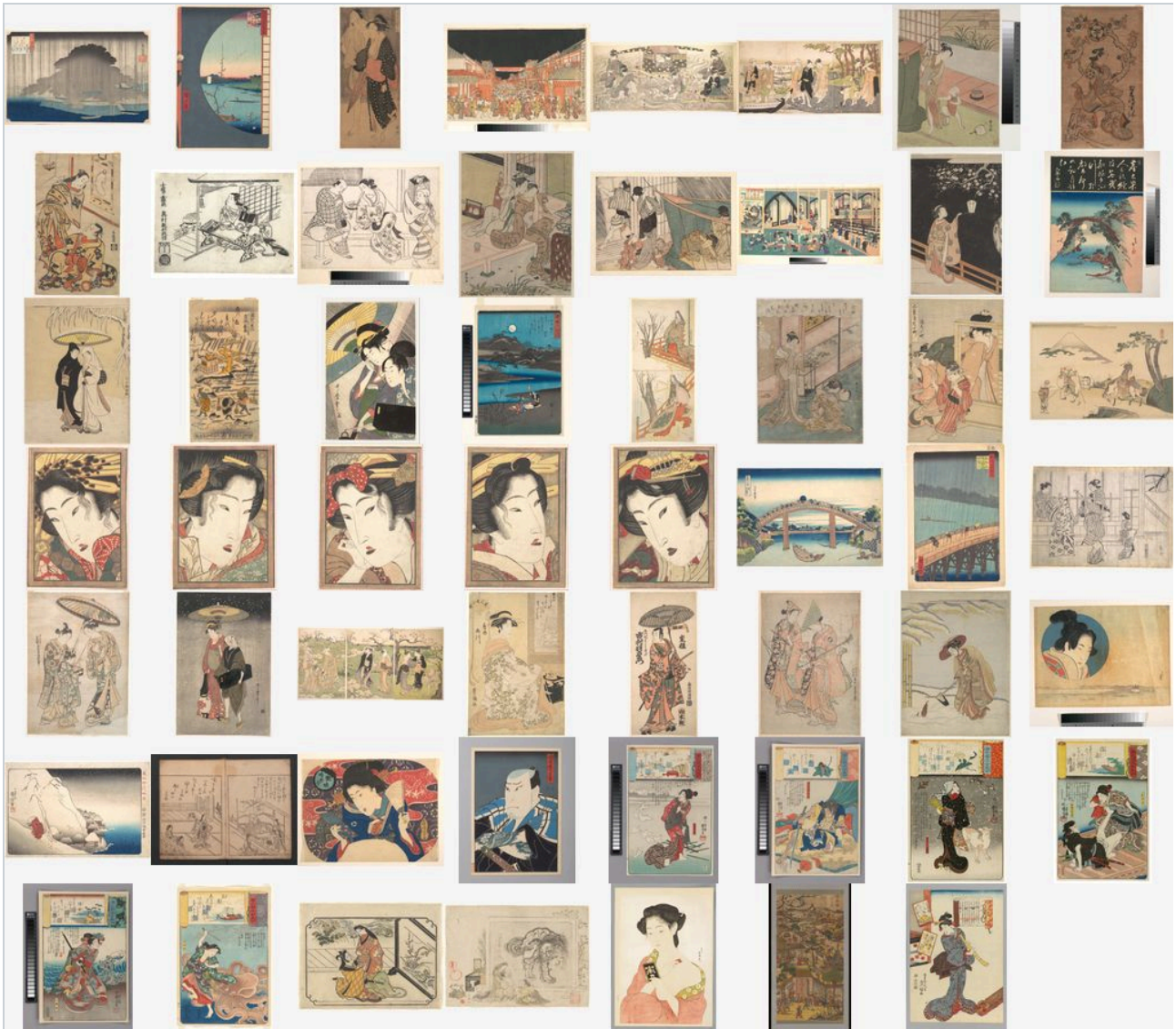


Figure 2. The ukiyo-e portion of dataset-v1, CC0 from the Metropolitan Museum of Art. Contact sheets were generated per style for visual inspection and are part of the evidence set rather than illustration.

<docs/evidence/dataset-v1/contact-sheet-ukiyo-e.jpg>

10.2 Licensing policy

The policy was written before any file was downloaded, and it is restrictive on purpose:

- **Permitted licences: public domain, CCo, project-original.** Nothing else enters the manifest.
- **Rejected outright:** stock-photo "free" sites, scraping, and AI-generated seed imagery.
- **Every item's licence and permitted use is recorded before it may enter a training split,** in a manifest carrying `source`, `author`, `licence`, `collection_date`, `permitted_use`, `dimensions`, `SHA-256` and `split`.
- **The concrete source registry required explicit human approval before any download.** No source was added to it by an assistant.

Exclusions are equally explicit: identifiable living persons, visible brands, logos or trademarks, NSFW content, contemporary artist-attributed works, watermarked images, and near-duplicates.

One style choice was made for licensing reasons and it is worth stating plainly. The original plan called for graffiti and street art. Graffiti photography is generally artist-copyrighted, and building a training set from it would have been either a licensing violation or an exercise in optimistic labelling. It was replaced with ukiyo-e woodblock prints, which have large, explicit, institutionally documented public-domain supply. The visual intent — bold, graphic, high-contrast source material — survived the substitution.

10.3 The pipeline

Nine stages, each scripted and tested: decode validation, SHA-256 hashing with exact-duplicate detection, perceptual near-duplicate detection, resolution and aspect statistics, a normalisation policy, caption tooling, deterministic seed-fixed split assignment, per-style statistics, and contact sheets for visual inspection.

Raw imagery is **not committed**. Manifests, statistics and contact sheets are. Model weights never are (§16.3).

The manifest is hash-locked and a test asserts it has not changed. During the style-learning milestone it was opened **read-only throughout**, so the claim that the dataset did not move during training is checkable rather than asserted.

10.4 Two findings that were recorded before they were convenient

Both were noticed while re-inspecting the poster contact sheet during a pre-check, **before any training run**, and both were recorded as open risks to the results rather than quietly fixed.

Framed and matted scans. Most `retro-poster` items are photographed or scanned with visible dark frames and cream mats. A style adapter could learn the frame as part of the style. A border-darkness measurement taken before training flagged **35 of 36** training items, with a median border delta of **-73.5**, against **+29.7** and 2 of 44 for ukiyo-e.

Text-dominated source material. Display typography is a defining feature of WPA posters, and diffusion models render text poorly. Only **14 of 36** poster captions describe anything visible; 20 of 41 are play-title attributions.

Both were confirmed to transfer. The style-learning gate marked unwanted framing and pseudo-text worse than baseline on **every** trained poster sheet, and that is why `retro-poster` ships as a partial pass rather than as an equal (§11.4, §18.1).

10.5 The mitigation that was chosen, and the one that was not

Two mitigations were available: crop the source material, or change the captions.

Captions were chosen. A style-only caption strategy was tested as a **blinded A/B changing exactly one variable** and selected by the student at the first review gate. The dataset was **never modified** — it remains byte-identical to its recorded hash and was opened read-only for the whole milestone.

The reasoning is recorded: a crop pass would have altered the dataset for one style only, breaking comparability with every earlier result, while the caption route was testable without touching a single source file.

What this does not establish, stated in the methodology rather than left implicit: the caption A/B ran on `minimal-geometric`, not on `retro-poster`. Style-only captions are therefore selected on evidence from the lead style plus the pre-training audit — **not** on a measured poster comparison. Removing frames at the pixel level was never tested and remains open (§23).

10.6 A correction to how these findings were first described

The initial write-up framed one reference image as the difficult case because it was framed and text-heavy. Opening the actual image showed that a second reference was **also** a framed, text-dominated poster scan; the first was harder only because it added landscape orientation, the one orientation a deck cannot accommodate.

The correction is recorded in the dataset methodology, and it made the problem **more** widespread than the original wording implied rather than less. It is included here because it is a small example of the rule the whole project runs on: check the artefact, not the description of it (§21).

11 Prototypes

Six prototypes were built in sequence. Each answered a question the next one depended on, and none was skipped. This section reports what each established; §12 reports what failed along the way, and §13 the measured results.

11.0 Prototype 0 — the 3D viewer

Question (RQ9): how is decal artwork correctly mapped onto a 3D deck — UV layout, nose-to-tail orientation, and runtime texture updates?

A procedurally generated deck mesh in React Three Fiber, with **asymmetric kicks** (nose 0.17, tail 0.12) so that orientation is physically meaningful rather than decorative, and a documented UV convention in which `v = 1` is the nose. Orbit, zoom and reset; runtime texture swap without reload; thirteen passing tests covering UV invariants, the nose mapping, winding, determinism and the kick asymmetry.

The hypothesis was confirmed on the first render, and the report says so rather than manufacturing a struggle. The decal landed correctly the first time. There is consequently **no fix commit for an orientation defect, because there was no orientation defect.**

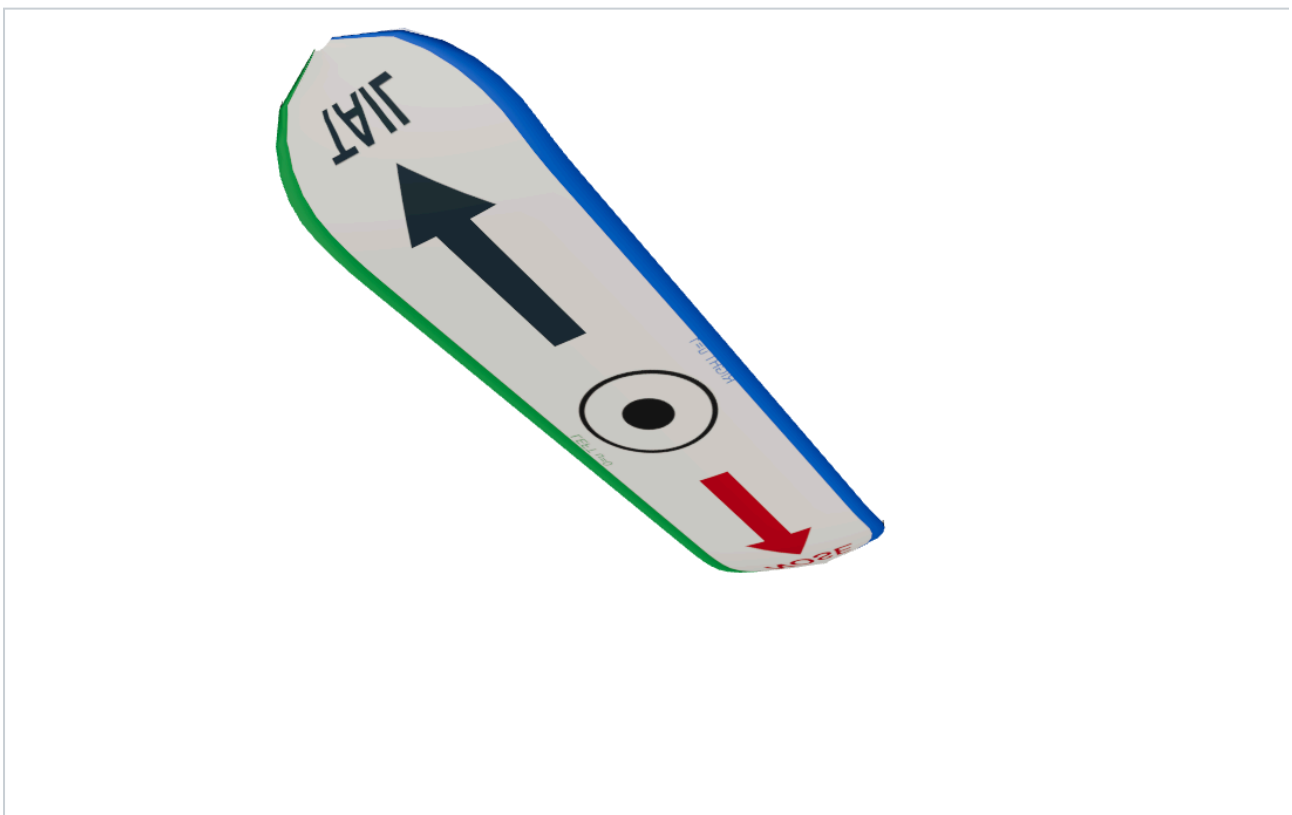


Figure 3. A controlled demonstration, not a record of a failure: a labelled developer toggle inverts the V axis so the consequence of a wrong UV layout is visible. The evidence files are named accordingly.

`docs/evidence/prototype-0/p0-02-inverted-uv-demonstration.png`

No experiment row exists for this prototype, because it contains no generative-model experiment. The registry begins at Prototype 1.

What it handed forward, including a trap. The test decals were 512×2000 , approximately 1:3.9 — close enough to the deck's UV domain that no mismatch was visible. That is why the geometry problem in §11.5 stayed hidden for four milestones.

11.1 Prototype 1 — base-model benchmark

Questions (RQ2, RQ8): which base model is feasible on 8 GB, and how should the deck's aspect ratio be produced?

Measured and argued in §9.1. Three results carried forward: **SD 1.5 selected, SDXL rejected on memory despite winning on quality**, and **direct 1:3 generation selected** — the last refuting its own hypothesis, which had predicted that tall generation would degrade composition.

11.2 Prototype 2 — text and reference conditioning

Question (RQ6): how should text prompting and a reference image be combined?

Four arms were compared on measured data: img2img, standard IP-Adapter, IP-Adapter-Plus, and a text-only baseline. ControlNet [7] was compared on criteria and **deliberately not implemented**, with the screen-out reason recorded.

Standard IP-Adapter was selected at a default scale of **0.55**, user-adjustable 0.40–0.60 (DR-008). IP-Adapter-Plus showed no decisive advantage at slightly higher memory cost and was not selected. The measured comparison is in §13.4; what follows is why the numbers decided it.

The result that decided it

img2img costs **exactly zero extra VRAM**, which on an 8 GB budget is a serious argument. It was rejected anyway.

Every one of the six near-copy flags in the milestone was **img2img at the deck format**, three of them at a perceptual-hash distance of 0–1 — visually indistinguishable from the reference image. The median distance for img2img at medium strength is 27 at 512×512 but **5 at 512×1536** .

The mechanism was identified rather than guessed: img2img forces the reference into the output resolution, so when the aspect ratio already matches, nothing is cropped and denoising begins from an essentially intact copy. **A method that returns its own input is not a generator**, and the student scored those outputs 1 for originality.

img2img is retained as a documented zero-extra-VRAM fallback, never as the default path.

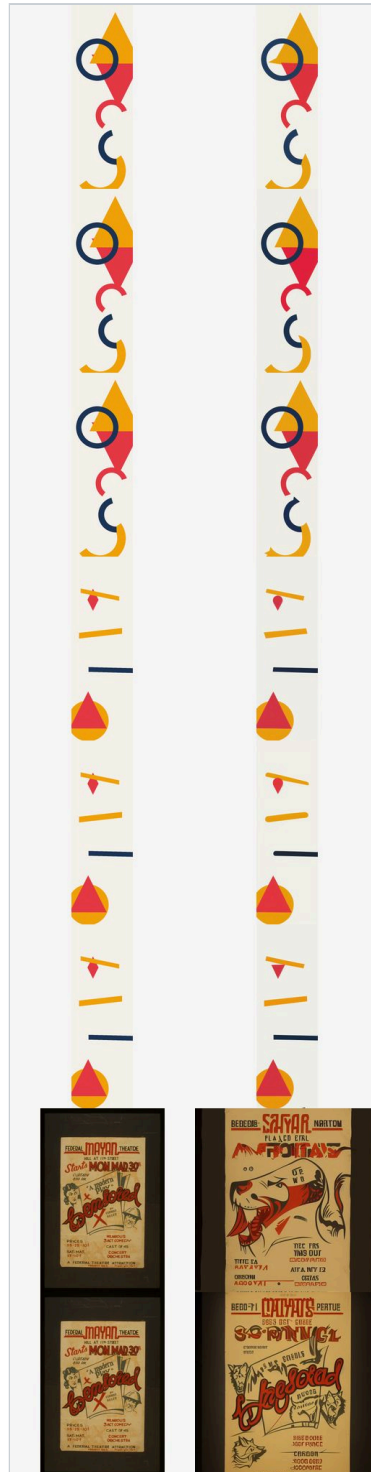


Figure 4. Reference images beside their img2img outputs at 512×1536. These pairs are why the zero-VRAM method was not selected.

[docs/evidence/prototype-2/copy-risk-pairs.jpg](#)

Two measurement decisions worth recording

The lower bound was tested, not promised. IP-Adapter at scale 0.0 produced output **byte-identical** to the text-only baseline in 12 of 12 runs, and that baseline was itself byte-identical to Prototype 1's — cross-milestone repeatability, measured.

The evaluation encoder was kept out of every process whose memory it would have inflated. CLIP similarity runs on CPU in a separate phase, and a test asserts the generation runner never imports it. Loading 2.35 GiB of metric encoder inside a generation process would have corrupted exactly the VRAM figures the comparison rests on.

11.3 Prototype 3 — LoRA smoke test

Question (RQ1): is local fine-tuning viable end to end on this machine?

Covered in §9.2. Thirteen runs, zero memory-tier escalations, LoRA selected (DR-009) at rank 8. The combined stack was proven to fit the deck format, which re-scoped the project's binding risk from *does this fit* to *does anything added to it still fit*.

Full record: `docs/prototypes/prototype-3.md`.

11.4 Prototype 4 — style learning

Questions (RQ4, RQ5): how much data and which caption strategy, and one multi-style adapter or several?

Ten training runs of a permitted twelve; both contingency slots went unused. Two human gates (§6.3).

What shipped

style	run	step	outcome
minimal-geometric	EXP-027	300	PASS
ukiyo-e	EXP-028	600	PASS
retro-poster	EXP-029	300	PARTIAL PASS

Three separate per-style adapters at a default weight of **0.7** (DR-010), each 6414480 bytes.

The finding worth remembering

Two of the three ship at step 300, not the 600 they were trained to. Prompt adherence fell from 4 to 3 at step 600 for both `minimal-geometric` and `retro-poster` while style consistency held at 5. Training longer made the model more stylish and less obedient. Only `ukiyo-e` improved.

That result exists only because checkpoints were saved at 150, 300, 450 and 600 and a human compared them per style, instead of assuming one global step count. It is also the origin of an accepted product limitation: **prompt adherence can be weaker than style adherence** (§18.1).

RQ5: multi-style is viable, and was not selected

The multi-style adapter was trained with **exposure asserted rather than hoped for** — an unbalanced run raises an error rather than producing a plausible adapter. It was competitive at 512×512 with no severe cross-style bleed.

It is not a failed experiment and this report does not present it as one. Per-style adapters won on flexibility, because each style turned out to need a *different* checkpoint step — which the multi-style approach cannot express.

RQ4: the caption A/B answered, the image count did not

Captions were compared as a **blinded A/B changing exactly one variable**, and style-only captions were selected at the first gate.

The image-count arm is **inconclusive** (§5.2): non-monotonic, no minimum established, equal compute rather than equal epochs, lead style only. The repetition confound was **declared in code before any result existed** — 25.0, 12.5 and 6.8 presentations per item — so the limitation is part of the experiment's design rather than an excuse added afterwards.

11.5 Prototype 5 — the integrated MVP

Question: the primary research question, end to end.

Covered in §14. Two results belong here because they are about the prototype ladder rather than the product.

A four-milestone-old assumption failed. Generated decals are 1:3; the deck's UV domain is 1:3.902. Prototype 0's 512×2000 test decals had concealed this since the first milestone. Both remedies were **built** rather than argued about — full-surface mapping with a 1.3008× longitudinal stretch, and fit-without-stretch leaving 23.12 % of the deck bare — each disclosing its cost numerically, with **a test asserting that no default was exported** so the choice had to be made by a human. The student selected full-surface (DR-012) and his rationale is quoted verbatim in the record rather than paraphrased.

The gate found what tests could not. Twelve manual acceptance items, all passing, plus three defects no automated suite had caught: a 70-pixel viewport overflow, a page-wide horizontal scrollbar from a CSS specificity conflict, and a polling loop restarting on every render.

11.6 What the ladder produced

Each prototype's output became the next one's input, and three times a prototype invalidated an assumption the previous one had left in place: Prototype 1 refuted the aspect-ratio hypothesis, Prototype 2 disqualified the cheapest conditioning method, and Prototype 5 exposed a geometry mismatch that four milestones of test assets had hidden. **A ladder that never contradicts itself is not being climbed carefully.**

12 Failed and blocked experiments

Failures are reported here as results, with the same detail as successes. Several of them changed the project's method more than any successful run did.

12.1 Blocked: three sources became unavailable mid-project

what	how it failed	what was done
Digital Comic Museum	Cloudflare gating	share shifted to an already-approved source
Art Institute of Chicago	image CDN returned 403	share shifted to an already-approved source
SD 2.1 base	HTTP 401 — repository gating	escalated to the student; two candidates used

The base-model block is the consequential one. Two sibling repositories from the same organisation also returned 401 while SDXL returned 200, which identified it as deliberate gating rather than an outage.

Nothing was substituted without approval. The student was asked and chose to proceed with two candidates. Three alternatives were declined and recorded, including an ungated community mirror rejected because its fidelity to the original cannot be verified while the original is gated.

The cost is permanent and is carried into the limitations: the base-model decision rests on two measured candidates rather than three, and that candidate cannot be reproduced today without authentication.

This pattern recurred often enough to be logged as a risk in its own right — third-party hosting becoming unavailable mid-project — and the mitigation adopted afterwards was to pin immutable commit hashes for every model actually used, and to verify availability at the moment of use rather than assume it.

12.2 The measurement that was contaminated, and the wrong explanation for it

An early aspect-ratio experiment produced a **20× timing spread for provably identical work**: one strategy took 7.96 s where a later run of the same work took 4.10 s.

The first hypothesis was thermal throttling. It was tested and ruled out — a hotter, more throttled card ran the same work *faster*. The actual cause was in-process CUDA caching-allocator state: all four geometries had shared one process.

Re-running one configuration per fresh process cut the timing spread roughly twentyfold, and **one-configuration-per-process became the measurement rule for the rest of the project** (§6.2). Every VRAM and latency figure in this report depends on that correction.

This is recorded as a failure because the first set of numbers was wrong and was published to the process log before the cause was found.

12.3 Defects found in the project's own tooling

Four are worth reporting, because each would have produced a plausible but false result.

An experiment runner crashed after loading the full stack. The combined-stack experiment's own runner unpacked one value from a function returning two. The failed row is **preserved in the evidence with `status: failed`** rather than deleted, and it still records that the full stack loaded at 308.33 MiB before failing in preprocessing. The defect was in the experiment, not in the stack.

Twenty-four duplicate generations were biasing a diversity indicator. Two blocks of the final matrix overlapped at one weight. The outputs were byte-identical, so each put a self-pair into a diversity cell and pulled it toward zero. The plan and the diversity pass were fixed and guarded by a test; **the matrix was not regenerated**, because its evidence is a valid superset of the fixed plan, and both fingerprints are recorded so the difference is inspectable.

A trigger-token design was rejected against the live tokenizer before any GPU time was spent. The student-approved plan's tokens split into four pieces, lost their shared prefix, or collided with words already in the caption corpus. Measured against the actual tokenizer, not assumed.

A reporting bug presented missing data as a passed check. Found in the analysis code, in the project's own favour, and corrected.

12.4 The reproducibility defect: training is not bit-reproducible

Diagnosed during style learning, and it is the project's most consequential internal failure.

The adapter is created with Gaussian initialisation drawing from the **global** torch RNG. The runner seeds a generator for the sampled latent, the noise and the timesteps — but never the global RNG. So every process starts from a different adapter.

It was **measured rather than inferred**. Same-step adapters from two runs differ by an L2 of about 158 against a weight norm of about 112 — a ratio of $\sqrt{2}$, the signature of two independent draws — while training itself moves the weights by about 5.

The impact is bounded, and the bound was checked. Each run's gates are self-contained. The data pipeline **is** deterministic and was verified so: a 600-step run's first 300 draws were byte-identical to the pilot's recorded sample-order hash. Every comparison was scored on generated images from arms differing in one recorded variable, never on weight equality.

It was deliberately not fixed mid-milestone. Seeding the initialisation would have altered every run the first gate's arms were compared against. The fix is forward-only, and the historical evidence stands unchanged.

The consequence is permanent: the three production adapters are authoritative **as files, by SHA-256, not as a recipe**. They cannot be regenerated. If they are lost, the production selection is lost with them (§16.3).

12.5 Five defects that only a clean clone could find

The testing milestone was scoped to verify and instead discovered. Every one of these was invisible on the machine the code was written on.

1. **A frozen dataset hash that had only ever passed locally.** The recorded digest was taken from a CRLF working copy while Git stores LF, so **an integrity control had never once been tested against a fresh checkout.** The fix separated the identifier from the content check, because repointing the original would have moved a fingerprint that earlier evidence cites as unchanged.
2. **A settings file documenting five variables nothing reads,** two implying upload security rules were configurable when they are not.
3. **A stale environment audit** — Node 20.18.0 recorded, 24.18.0 actual, with no record of when it changed.
4. **A requirements file claiming a pin it did not make.** Four "pinned" lines were comments.
5. **--workers 1 starts two processes,** a supervisor and a worker, so the health endpoint reports a process ID the launcher never recorded and a naive stop strands the worker holding the port.

The first is the most instructive finding in the project. **Nothing in the risk register anticipated "the check itself is wrong."** A clean-clone test is now the control for that class of defect (§16.2).

12.6 The continuous-integration failure, and what it cost to fix

The first remote run failed one browser scenario that passed locally: a camera-preservation test timed out after 300 000 ms on the runner.

It took five runs. The scenario was rewritten twice — screenshot comparison, then structural with host-side polling, then structural, in-page and frame-counted — and **passed remotely on its first attempt in 40.2 s.** Two defects in that work were found on the way, one of them caught by a unit test before it reached the runner: an equality rule for a camera that damping means never comes exactly to rest, and a drift tolerance wide enough to swallow an entire change of viewpoint.

The application was never changed. Both rewrites were of the *measurement*.

The remaining instability was the runner itself. Failures moved between runs — 1 failure, then 3, then 6, and a *different* six — with a 22× swing on identical code. A trace showed a single mocked response taking 12.78 s against a 10 s expectation, and 59.1 s of a 60 s budget consumed before the assertion under test began. Three neighbouring scenarios with identical setup completed in 5.6, 8.0 and 9.1 s while the same code path took over two minutes in between.

Retries alone cannot fix a stall that outlasts a retry cycle, and the evidence shows exactly that: one scenario failed all three attempts inside a single stall window while another's retry landed outside it.

CI budgets were raised for the runner only, on the traced evidence rather than by taste, and local budgets were deliberately left alone so the suite keeps its performance signal.

How the resulting green must be read, and this report states it rather than claiming a clean pass:

- a green run also occurred under the **old** budgets, so the green cannot be attributed to the budget change;
- the **per-scenario retry counts of the final green retry-enabled run are unknown, not zero** — the collapsed log did not carry them;
- **a green under two retries and a 180 s budget is weaker evidence than a first-attempt green under 60 s.**

What stands without qualification is the camera scenario's **first-attempt 40.2 s remote pass, before any budget was raised.**

The raised budget is not described as a fix. The stall is real and unexplained, and a budget that lets a slow environment finish also costs the suite its ability to detect a genuine performance regression.

12.7 The lesson these share

Three of the six subsections above are defects in **verification** rather than in the product: a contaminated measurement, an integrity check that had never run anywhere but its author's machine, and a test that measured the wrong thing twice. That is the pattern worth carrying forward, and §18.4 records what it costs the evidence in this report.

13 Experiment results

13.1 The registry

40 experiments are registered in `experiments/registry.csv`, each with its research question, hypothesis, dataset version, model and pinned revision, full configuration, hardware readings, duration, output paths, evaluation, conclusion, next action and commit.

Three rules governed the registry, and they are why it can be cited rather than merely referenced:

- **A row is created when the experiment runs**, never retroactively.
- **Memory and duration are measured, never estimated.** Unmeasured means the row says *not measured*.
- **Failed experiments get rows too.**

prototype	experiments	principal questions
1	EXP-001...005	environment gate, base model, deck aspect ratio
2	EXP-007...014	conditioning method, controllability, near-copy risk
3	EXP-016...019	LoRA feasibility, deck-format training, combined stack
4	EXP-020...033	style learning, captions, image count, memorisation
5	EXP-034...035	service residency, reference neutralisation

Two identifiers are absent by design: EXP-006 and EXP-015 name the human scoring directories rather than runs.

13.2 Memory, measured across five milestones

This is the project's spine, and its internal consistency is the strongest evidence that the measurement protocol worked.

configuration	geometry	peak allocated	peak device	spare
SD 1.5 text-only	512×512	2 675.38	4 359.5	3 828.0
SD 1.5 text-only	512×1536	3 892.01	—	—
SDXL	512×512	7 859.26	8 187.5	~0
SDXL	1024×1024	10 738.08	8 187.5	overflowed
+ IP-Adapter	512×512	3 924.07	5 695.5	2 492.0
+ IP-Adapter	512×1536	5 140.69	7 965.5	222.0

configuration	geometry	peak allocated	peak device	spare
LoRA training	512×512	3 133.40	4 285.5	3 902.0
LoRA training	512×1536	5 182.58	6 449.5	1 738.0
+ LoRA + IP-Adapter	512×1536	5143.73	7 985.5	202
the same, serving	512×1536	5143.73	—	200

MiB, against 8187.5 MiB physical.

Three consistency results fell out of this table without being sought:

- **The text-only baseline is byte-identical across three milestones** — 2 675.38 MiB in Prototype 1, again in Prototype 2, and again as the reference point in Prototype 3.
- **Peak allocated for the production stack is byte-identical across three measurements** taken days apart, in different milestones, including one inside a freshly built clean clone.
- **A trained adapter costs +3.04 MiB regardless of output geometry**, measured independently three times.

A measurement protocol that reproduces to the byte across milestones is one whose comparisons can be trusted. That is the argument for one-configuration-per-process, stated as a result rather than as a principle.

13.3 Training results

Ten runs, all at the lowest memory tier, none escalated. Six were pilots comparing captions and set size; the four that produced or tested production candidates are below. The full set is indexed in Appendix A.

run	style	steps	s/step	wall	first → last loss
EXP-027	minimal-geometric	600	0.283	175.9 s	0.0780 → 0.0025
EXP-028	ukiyo-e	600	0.398	244.1 s	0.6583 → 0.2297
EXP-029	retro-poster	600	0.308	189.5 s	0.4973 → 0.1425
EXP-030	multi-style	1800	0.350	633.8 s	0.6290 → 0.1960

Peak allocated was 3 133.4 MiB in all ten runs. Geometry sets training memory; style, image count and step count do not. `ukiyo-e` is slowest per step because its source images reach 4 000 pixels — a data-loading cost, not a model cost, and identified as such rather than left as an anomaly.

Loss is reported and not interpreted as quality. The lowest final loss belongs to `minimal-geometric`, whose sources are synthetic and therefore easiest to fit. It is not the best style.

13.4 Conditioning results

Both methods gave **monotone, human-visible control in 6 of 6 conditions**, clearing the pre-declared bar. The differences that decided the selection were elsewhere.

	img2img	standard IP-Adapter
extra VRAM	0	+ 1 248.69 MiB
latency behaviour	falls with strength	flat across scale
usable mid-range	0.60–0.65	0.40–0.60
near-copy flags	6 of 6 in the milestone	0
originality score at the deck format	1	4

The latency advantage is an artefact and is reported as one. Diffusers [15] runs `int(steps × strength)` steps, so a stronger reference is also a shorter run; cost per *effective* step is flat at 0.11–0.13 s for both. Reading the wall-clock figure alone would have credited img2img with a speed-up it does not have.

13.5 Evaluation and memorisation

0 of 252 final-matrix outputs were flagged as near-copies of training images, with a holdout control at a comparable distance — which is the point of the control.

WHAT THIS DOES NOT PROVE

A perceptual-hash threshold is a **coarse near-copy indicator, not proof of no memorisation**. It detects near-duplicates, not learned reproduction of style-level structure. The CLIP similarity measure has a second limitation: it uses the same model family that IP-Adapter conditions on, making it descriptive *within* a method rather than a neutral referee *between* methods.

13.6 Service results

The production stack was run as a long-lived service — twelve requests, six cases twice, swapping between all three adapters.

- **Allocated memory after generation: 3 316.64 MiB in all 13 runs**, growth 0.00 MiB against a 64 MiB allowance. Residency itself costs nothing.
- **Worst spare: 200 MiB**, tighter than the one-shot figure, and the operative production ceiling.
- **Byte-identical repeats** across cycles — strong evidence of no state residue, and explicitly not proof of none.
- A corrupted adapter copy produced a 503 and clean recovery with no restart; a deadline exceeded produced a 504 after 14 of 30 steps with the lock released.

This experiment **deliberately breaks the project's own one-configuration-per-process rule**, because serving is the thing being measured. Its record says so, and states that its figures are therefore **not comparable** with the single-shot experiments.

13.7 The result that closes the loop

A clean clone, in a freshly built environment three days later, reproduced an earlier output **byte for byte** (§16.2): SHA-256 `46bbf160e4270429e6692467dc6c59577e99bf3178dedd8d38193d0335fb6d7f` , 1089939 bytes.

Inference is deterministic and portable given a fixed adapter — which does **not** contradict training being irreproducible from seed (§12.4). Different halves of the pipeline; both true.

14 The integrated MVP

14.1 What it does

A customer enters a prompt, optionally uploads a reference image, picks one of three styles, adjusts reference strength and LoRA weight within bounded ranges, and generates. The result appears as a 2D decal and on an interactive 3D deck that can be rotated, zoomed and reset, and both the artwork and the deck view can be downloaded.

```
React + Three.js  —HTTP—>  FastAPI (one process, one worker)
                             |
                             |— single-flight lock (process-local)
                             |— resident pipeline
                                SD 1.5 @ 451f4fe1
                                + one of three style LoRAs @ weight 0.7
                                + IP-Adapter [11] @ 018e4027, scale 0.55
                                -> 512x512, 30 steps, guidance 7.5, DPM-Solver++ [9]
```

Generation is direct to the deck format. The pipeline loads on the **first** request, so that one takes about 30 s and every later one 12–13 s.

14.2 One process, one worker — a measurement, not a preference

The service holds one pipeline resident and serves one generation at a time behind a process-local lock. A startup guard rejects a worker count above 1.

This follows from §9.3 arithmetic: the production stack leaves **200 MiB spare**, and a second resident pipeline needs about 5 GB. The lock being process-local means a second API process would not see it, so **multiple separately launched processes are unsupported and undetectable from inside one**.

STATED AS A LIMITATION, NOT A VIRTUE

Scaling this system is **not a configuration change**. It requires a second GPU or a smaller resident footprint. The single-process design is what the memory measurement permits.

The lock's discipline is tested rather than assumed: it is released only after the generation call returns, a second request is refused while work is active, a later request succeeds after a controlled abort, and a client disconnect does not free it.

14.3 Progress reporting that refuses to invent a number

The interface reports real progress, and the design was constrained by one instruction from the student: *this must not be a fake progress bar*.

Only denoising has a real denominator. So the implementation publishes a step count during denoising and, for model loading, decoding and saving, a **stage name with a null estimate**. There is no weighted overall percentage, because no honest one exists.

- A test enumerates every non-denoising stage to assert **no percentage is produced** for it.
- **100 % waits for the PNG to decode in the browser**, not for the last diffusion step to land.
- The "finalising" stage is genuinely about a second and **is deliberately not padded** to make the label linger. Padding a finished result is exactly the dishonesty the feature exists to avoid.

This replaced a static "around 15 seconds" line that was wrong for a cold start by a factor of two.

14.4 The geometry decision

Generated decals are 1:3. The deck's UV domain is 1:3.902. Something has to give, and Prototype 0's 512×2000 test assets had hidden the discrepancy for four milestones (§11.5).

option	cost	status
full-surface	1.3008× longitudinal stretch	production default (DR-012)
fit-without-stretch	23.12 % of the deck bare	retained and selectable
regenerate at 512×1998	invalidates every VRAM figure	screened, not measured
reshape the deck to 1:3	invalidates Prototype 0's evidence	screened, not measured

Both shipped options were built, neither was argued for, and a test asserted that no default was exported so the choice had to be made by a human looking at two screenshots differing in one variable. The student selected full-surface, and his rationale is quoted **verbatim** in the decision record — a justification the assistant invented would have been worse evidence than none.

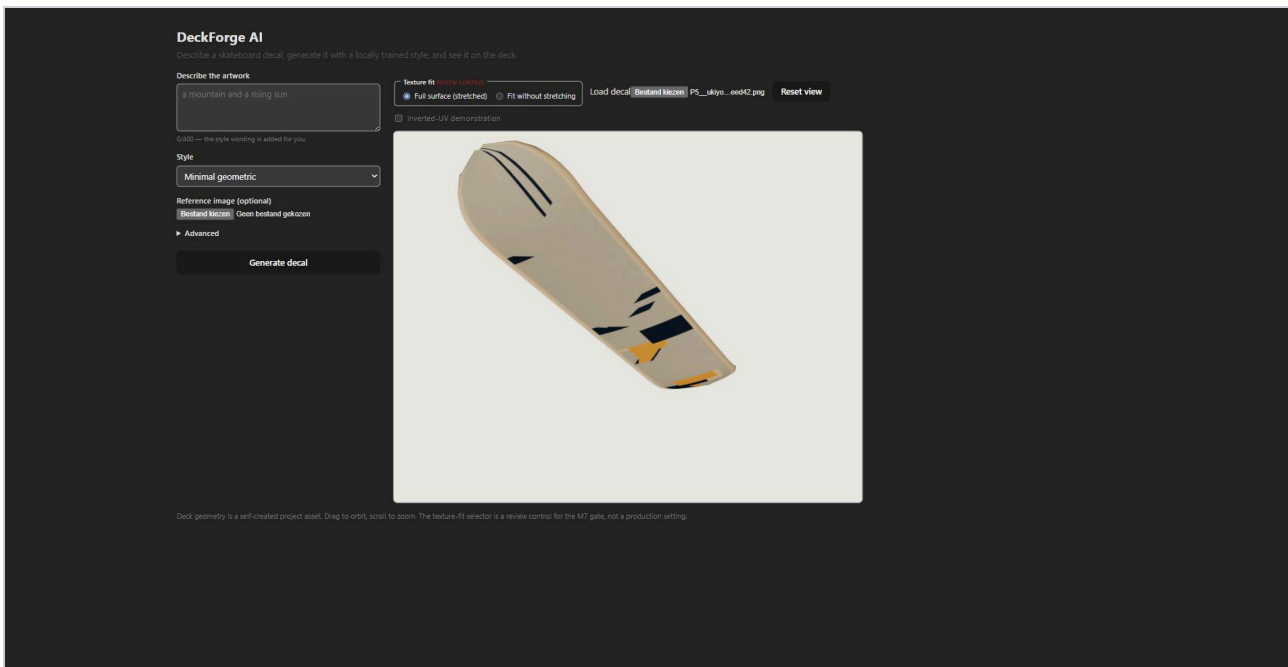


Figure 5. The selected mode: the decal covers the whole deck surface at the cost of a 1.3008× longitudinal stretch. The rejected alternative is kept selectable because it is the evidence behind the decision.

<docs/evidence/prototype-5/screenshots/fit-full-surface.jpg>

The evidence behind DR-012 is **one reviewer, one decal, one camera**, and the record scopes the judgement to the three production styles at the deck format rather than claiming that a 1.3× stretch is imperceptible in general.

14.5 A feature added at the review gate

Upload your own decal was scoped by the student after using the application: customers who already own artwork should not have to spend a generation to see it on a board.

It is **wholly local** — decoded in the browser, never sent to the server, never touching the model — and that claim is proved from **server-side request counts** rather than asserted. It is kept visually distinct from the AI reference upload so the two cannot be confused, uploaded artwork receives **no reproducibility metadata** because none exists for it, and a failed decode preserves the previous texture.

It also turned out to be the load-bearing element of the demonstration fallback plan, because it needs no GPU, no model and no server round trip (§16.4).

14.6 Reproducibility surfaced to the user

Each result carries the model revision, adapter, weights, scale, seed, steps, guidance and geometry, and the interface shows them. The `retro-poster` limitation is surfaced **to the user on every request** rather than confined to documentation.

Metadata never contains a filesystem path, and generation identifiers resolve through a registry lookup rather than a path join — so a traversal string has nothing to traverse (§17.3).

14.7 What the MVP does not do

It does not batch, queue, or serve concurrent users. It has no accounts, payments or persistence beyond the generation registry. There is no CPU fallback: without a GPU and the three adapter files, the service returns 503 for every style, by design and verified in both directions (§16.3).

15 Testing

15.1 The suites

suite	count	scope
pytest (<code>ml</code> + <code>apps/api</code>)	473	dataset tooling, ML inference and training tooling, API contracts, upload security
vitest (<code>apps/web</code>)	183	API client, form validation, decal provenance, texture-fit geometry, texture swapping
Playwright (Chromium) [20]	38	the built frontend, every <code>/api/**</code> call answered from frozen fixtures
eslint	—	clean
production build	—	succeeds

No Python linter is installed, so pytest is the Python gate, and this report says so rather than implying a linting step that does not exist.

A clean clone runs 522 passed and 5 skipped, re-measured in the M11 audit on 2026-08-15. The five are pre-existing conditional skips for git-ignored assets, each declaring its reason; $522 + 5 = 527$, matching the development machine exactly. That figure is a whole-repository run, so it is larger than the 473 system tests this chapter is scoped to: it also contains the 16 report-validation and 38 deck-validation tests, which test the documents rather than the application. The M8 measurement of $468 + 5 = 473$ was correct for its date and is superseded here, not contradicted.

15.2 What the suites deliberately do not prove

THE MOST IMPORTANT SENTENCE IN THIS SECTION

No test in any of the three suites loads the model or runs a generation. They prove the code, the contracts, the upload rules and the interface. **A green suite is not evidence that DeckForge AI generates anything.**

Generation quality, memory, latency and adapter integrity under real serving are evidenced separately: by the experiment registry, by the residency and neutralisation experiments, by a validation script run against a real server process, and by the single authorised clean-clone generation. **Neither kind of evidence is presented as the other**, and the testing strategy states this in its own opening rather than leaving a reader to infer it.

15.3 How the layers were designed to catch different things

The API layer runs with the pipeline stubbed — no GPU, no model, no network — so it can test every status code, the lock discipline, the adapter lifecycle across all three styles and back, and the checkpoint integrity gate against missing, wrong-size and **same-size-wrong-content** adapters.

The browser layer runs against the built frontend, not a dev server, with every API call answered from frozen JSON fixtures.

That last choice creates a risk the project took seriously: **a mocked suite can keep passing against a shape the backend no longer produces**. So the fixtures are validated against the real Pydantic models by a Python test. A backend field rename therefore breaks a Python test rather than leaving the browser suite passing against a stale contract — the mock cannot rot into theatre.

Some tests exist to stop a specific past mistake returning. A style-label regression guard asserts that the corrected style name cannot revert. A test asserts the training and inference memory ladders stay distinct, so "tier 2" cannot come to mean two things. Import boundaries are **AST-parsed rather than text-scanned**, in both directions. A guard asserts that gradient accumulation never appears as a memory tier — the correction the student made in plan review (§21.1).

15.4 Tests that encode research decisions

Several tests exist because a research decision would otherwise be a comment that drifts.

- The **frozen prompt kit and smoke kit are hash-locked**, so a comparison cannot be invalidated by an edited prompt.
- The **human scoring artifacts are hash-locked** and pinned in `.gitattributes` against line-ending rewriting, which makes "no score was edited after unblinding" checkable.
- The **holdout-exclusion proof joins against the dataset** rather than against the training manifest's own split column, so the manifest cannot vouch for itself.
- **Unmeasured sentinels must never satisfy a gate**, and `loss_decreased` must never gate anything.
- The **weights manifest is asserted against the code** by a test, so the documented adapter paths, sizes and hashes cannot drift from what the service actually loads.

15.5 Upload security

Twenty-seven rules, each with at least one negative test: extension and MIME allowlisting, mismatch between MIME and extension, undecodable bytes, truncated files, oversize input, **decompression bombs**, traversal filenames, and a GIF renamed to `.png`.

Generation identifiers resolve through a registry lookup, never a path join. This is the structural half of the defence: a traversal string has nothing to traverse, rather than being sanitised and hoped about.

The security matrix also records **what is not tested**, which is the half of a security document that usually goes missing.

15.6 Continuous integration

The workflow runs the Python suite, the frontend suite with linting and build, and the browser suite, on a GPU-less runner.

All three jobs pass — under three qualifications that materially weaken what the green means, set out in full in §12.6 and carried as validity threats in §18.4. They are not repeated here, because a green badge invites over-reading and a summary of a qualification tends to lose it.

CI depends on an external host for a tokenizer, so a red build with no defect behind it is possible. That is recorded rather than worked around.

15.7 Growth, and one restraint

The suites grew from 406 pytest and 165 vitest at the MVP's close to 473 / 183 / 38.

One planned addition was **dropped after checking disproved its premise**: a skip-marker for checkpoint tests on CI turned out to be unnecessary, because the suite already passes with the weights removed. Two further planned tests were dropped as duplicates of existing coverage. Adding any of them would have inflated the count while implying coverage that had not changed.

A test count is only evidence if every test earns its place.

16 Deployment and reproducibility

16.1 The decision

Three options were compared (DR-014):

option	verdict
A — native local, two processes, plus backup demo assets	selected
B — Docker Compose with GPU passthrough	screened out, not benchmarked
C — cloud GPU	rejected

Docker was screened out rather than measured, and the record says so. Its GPU overhead against the 200 MiB margin is unmeasured, measuring it would have cost generations the closed budget did not have, GPU passthrough was never verified on this machine, and the container toolkit is not installed. It would also not solve the actual reproducibility problem, which is the three unregenerable adapter files, not the Python environment.

Cloud GPU was rejected on a research ground as much as a cost one: different hardware would invalidate every VRAM figure in this report.

Running it is three commands — a preflight with ten checks including the three adapter hashes, a start script, and a stop script that stops the process **tree** rather than a recorded PID, because `--workers 1` starts two processes (§12.5).

16.2 The clean-clone test

The runbook was validated by cloning the repository into a fresh directory, building the environment from scratch, and running the system — eighteen ordered steps with real output recorded.

It failed, and the failure was the point. A frozen dataset hash did not match, because it had been recorded from a CRLF working copy while Git stores LF. An integrity control had never once been tested against a fresh checkout (§12.5).

The fix was **taken to the student rather than applied**, because repointing that constant would have moved a fingerprint that an earlier milestone's evidence cites as unchanged. The chosen resolution separates the M6 *identifier* from the *content check*, and a test now asserts the two stay distinct.

After the fix, the clean clone completed and produced **one authorised real generation**, which reproduced the earlier output **byte for byte** — SHA-256

`46bbf160e4270429e6692467dc6c59577e99bf3178dedd8d38193d0335fb6d7f` , 1089939 bytes, three days later in a freshly built environment.

Inference is deterministic and portable given a fixed adapter. Training is not (§12.4). Both statements are true and neither weakens the other.

16.3 The weights problem

The three production adapters are **6414480 bytes each** and are **not in Git** — model weights are never committed. They also **cannot be regenerated**, because training is not reproducible from seed.

This makes them the project's single point of failure, and it is mitigated structurally rather than by care:

control	what it does
weights manifest	records exact path, byte size and SHA-256 per adapter; asserted against the code by a test so it cannot drift
<code>scripts/verify-weights.ps1</code>	restores from a parameterised source and verifies, failing loudly per style
runtime verification	the service re-verifies each adapter's SHA-256 on every style activation , not once at startup

The integrity gate was proved in both directions — 3 of 3 failing before restore, 3 of 3 passing after — and it was **never proved by damaging a real adapter**. A corrupted *copy* was used, because the originals cannot be regenerated.

THE RESIDUAL RISK, STATED PLAINLY

The clean-clone test restored the weights **from the working repository, not from the external backup drive**, which was unavailable. The restore *mechanism* is validated; the **external backup itself is not**. If the working machine and that drive are both lost, the production selection is lost with them.

16.4 Demonstration readiness

A four-minute timed demo script exists, built around talking over the generation rather than padding it, and front-loading the cold-load wait as explanation rather than silence.

The backup plan has four rungs, each executable in under thirty seconds. The load-bearing one is that **Upload your own decal needs no GPU, no model and no server round trip**, so a dead pipeline costs about forty seconds of a four-minute talk and the entire 3D section runs unchanged. That is also why the feature appears at 2:30 in the main script: using it as a fallback then reads as continuity rather than retreat.

Backup assets are **existing validated material only** — real generated decals, the interface screenshots, the deck and clean-clone captures — assembled by a script into a git-ignored directory with a committed manifest.

One asset does not exist: a screen recording. It is listed as missing rather than assumed, because it must be a recording of a real session or it does not exist. A machine failure with no projector is **not mitigated**; that falls to slides.

16.5 What "reproducible" means here, precisely

The word is used in this report in three different senses, and conflating them would overstate the result.

sense	status
The environment reproduces from a clean clone	yes — validated, 18 steps, real output
Inference reproduces byte-for-byte given a fixed adapter	yes — measured across machines and days
Training reproduces from a recorded seed	no — the adapters are artifacts, not recipes

The third is a genuine limitation of this project's research output, and §18.2 carries it rather than letting the first two stand for it.

17 Ethics, copyright, privacy and bias

17.1 Training data and copyright

Generative image models are contested precisely because of what they are trained on. This project cannot resolve that debate, but it can be answerable for its own dataset, and it was designed to be from the first day.

Every one of the 148 items is public domain, CCo, or created for this project. No item entered a training split before its licence and source were recorded. Stock-photo "free" tiers, scraping and AI-generated seed imagery were rejected as source categories, and the source registry required human approval before any download.

The distinction that matters: this is a claim about **what was collected**, and it is verifiable per item from the manifest (§10.2).

THE LIMITATION THIS DOES NOT ESCAPE

The **base model** was not trained by this project. Stable Diffusion 1.5 was trained on a large web-scraped corpus whose licensing is the subject of active dispute and litigation. A clean fine-tuning dataset does not make the base model's provenance clean, and no amount of care in §10 changes that. **The output of this system inherits the base model's provenance**, and any commercial deployment would have to confront it directly.

There is a second inheritance. The model card for the base weights states that the model "is intended for research purposes only" [10]. This project is research and sits inside that intent, but the assignment frames a commercial client, and a real deployment would need a licence position this report is not in a position to give.

17.2 Reproducing the reference image

A reference-conditioning feature can become a copying machine, and in this project it demonstrably did — the measured result is in §11.2.

The ethical reading of that result is what belongs here. Had img2img been selected for its zero memory cost, the system would have offered users a route to reproduce any image they uploaded. That is a problem about what a product invites people to do, and it would have existed regardless of whether anyone noticed the perceptual-hash flags. **The method that was rejected was the cheaper one**, and it was rejected on this behaviour rather than on quality.

Flagged outputs are surfaced rather than deleted, and the rejected method survives only as a documented fallback with its behaviour stated.

Memorisation of training data was checked separately and found none — which, as §13.5 states, is a coarse indicator rather than proof.

17.3 Privacy and untrusted input

In the dataset: no identifiable living persons, no personal data in filenames or metadata, and historical artworks only from institutional public-domain collections.

In the running system: every upload is untrusted input reaching an image decoder, and is treated as hostile. Extension and MIME allowlisting, mismatch detection, actual decode validation, size and dimension limits, decompression-bomb rejection, random internal filenames, and traversal prevention — twenty-seven rules, each with at least one negative test (§15.5).

The structural decision matters more than the rules: **generation identifiers resolve through a registry lookup, never a path join.** A traversal string has nothing to traverse, rather than being sanitised and hoped about. Errors are safe by design — no local paths, no stack traces — and metadata never contains a filesystem path.

Uploaded decals never leave the browser. The `Upload your own decal` feature decodes locally, sends nothing to the server and never touches the model, and that is proved from server-side request counts rather than asserted (§14.5).

17.4 Bias

Three biases are present in this system and are named rather than managed away.

The dataset is culturally narrow, and unevenly so. Two of three styles are Western institutional archive material; the third is Japanese woodblock print, collected from a Western museum's digitisation of it. A model trained on 55 ukiyo-e prints from one institution's holdings learns that institution's collecting history — what it acquired, from whom, and what it chose to digitise — not the tradition itself.

One style is synthetic. `minimal-geometric` was generated programmatically by this project, so it contains exactly the biases of the generator that made it: six palettes, six shape counts, and nothing else. It is the easiest style to fit, and §13.3 notes its lowest final loss is a property of that synthetic simplicity rather than a quality result.

The base model's biases are inherited wholesale and were not measured. Diffusion models trained on web-scale data carry well-documented representational skews. This project ran no bias evaluation of generated output, and **does not claim the system is unbiased** — it claims only that the fine-tuning data's composition is documented and inspectable.

17.5 Honest representation of what the system is

Two design decisions in this project are ethical positions about not misleading a user, and both cost something to hold.

The progress display refuses to invent a number. Only denoising has a real denominator, so loading, decoding and saving publish a stage name and no percentage, and the one-second finalising step is not padded to make the label linger (§14.3). A weighted fake percentage would have looked better.

A weaker style ships with its weakness visible. `retro-poster` is a partial pass, and the application warns on **every request** for it rather than confining the caveat to documentation (§14.6). The same applies to the prompt-adherence limitation, which was found during a walkthrough and preserved rather than hidden by rewriting user prompts behind their back.

17.6 What was not addressed

- **No content filter.** The safety checker is present but disabled, as is conventional for research use; a deployed system serving customers would need a moderation position this project does not take.
- **No provenance marking.** Generated images carry reproducibility metadata but no watermark or C2PA-style content credential.
- **No bias evaluation of output**, as above.
- **No consent mechanism** for uploaded reference images beyond the fact that they are processed locally in the browser for decals and transiently for conditioning.

Each is a real gap. They are listed here rather than in §23 because they are not features that were postponed — they are questions a commercial deployment would have to answer before launch.

18 Limitations

Limitations are separated into four classes, because they have different consequences: a product limitation affects a user, a research limitation affects what may be concluded, a deployment limitation affects whether the system can run elsewhere, and a validity threat affects whether the evidence means what it appears to mean.

Eight of these were approved as limitations at a review gate, before this report existed, so they could not be reclassified retroactively to suit the writing.

18.1 Product limitations

1. **Cold model loading has no honest percentage.** The pipeline loads on first request and only denoising has a real denominator, so loading publishes a stage name and no number (§14.3).
2. **The time estimate is approximate and covers measured denoising only.**
3. **"Finalising" may be visible only briefly** — about a second — and is deliberately not padded.
4. **Prompt adherence can be weaker than style adherence.** A prompt for a futuristic city skyline with a skateboarder produced a clearly on-style, usable deck graphic **in which the skyline and the skateboarder were not clearly represented**. Strong style conditioning dominates detailed content. This is consistent with the measured drop in prompt adherence at step 600 (§11.4) and is **not** a frontend defect. It was preserved rather than answered by silently rewriting prompts.
5. **retro-poster ships as a PARTIAL PASS.** It learned the frames and display typography of the archive posters it was trained on — confirmed, not suspected (§10.4) — and warns on every request.
6. **One generation at a time.** One API process, one worker, one resident pipeline.
7. **No CPU fallback and no container.** Without a GPU and the three adapter files the service returns 503 for every style, by design.

18.2 Research limitations

1. **RQ4's image-count result is inconclusive.** Non-monotonic ordering, **no minimum image count established**, matched on **equal compute rather than equal epochs**, and measured on **minimal-geometric only**. It does not generalise to the other two styles.
2. **DR-009 selects LoRA on feasibility, not superiority.** From-scratch training, full fine-tuning, DreamBooth [5] and Textual Inversion [6] were **screened on criteria and never executed**. No claim is made that LoRA is the best of the five.
3. **The base-model decision rests on two measured candidates.** SD 2.1 was gated behind authentication (§12.1) and cannot be reproduced today without credentials.
4. **Rank and learning rate were set, not swept.** Rank 8 and 1e-4 were fixed at the smoke-test stage. RQ7 is answered for reference strength and adapter weight only.
5. **Training is not bit-reproducible from its recorded seed (R14).** The three production adapters are authoritative **as files, by SHA-256, not as a recipe**, and cannot be regenerated

(§12.4).

6. **Zero near-copy flags is not proof of no memorisation.** A perceptual-hash threshold is a coarse indicator (§13.5).
7. **One human approver, and AI-assisted visual analysis.** ChatGPT proposed scoring of the contact sheets and Kylian Algoet reviewed and approved the recorded scores (§6.3). There was **no second independent human rater**, so no inter-rater agreement can be reported, and the AI's proposals are a further influence on the recorded values that fixed anchors and blinding limit but do not remove.
8. **The Gate-2 sheets were labelled**, and labelled sheets carry an expectation effect the blinded Gate-1 sheets did not. Recorded at the time rather than noticed later.
9. **The caption A/B ran on the lead style, not on the style whose captions were the problem** (§10.5).
10. **No long native training run at the deck format was performed.** The 512×1536 training arms are feasibility probes of 1 and 10 steps; they establish cost, not style quality.
11. **The similarity indicator uses the same encoder family that IP-Adapter conditions on** [8], making it descriptive within a method rather than neutral between methods.

18.3 Deployment limitations

1. **The production margin is 200 MiB** of 8187.5 MiB — 2.4 % of the device. **This is not comfortable headroom.** Anything added to the stack has to fit inside it.
2. **Validated primarily on one GPU**, an RTX 4060 Laptop. The figures in this report are that machine's.
3. **Scaling is not a configuration change** (§14.2).
4. **Docker was screened out, never benchmarked.** Its GPU overhead against the margin is unmeasured (§16.1).
5. **The external weight backup was never exercised.** The clean clone restored from the working repository; the **mechanism** is validated, the **external drive is not** (§16.3).
6. **Three pre-existing high-severity npm advisories** remain, in dev tooling only, deliberately unfixed because the remedy moves the whole validated frontend toolchain under a freeze.
7. **Transitive Python versions are unpinned.** The misleading comment was fixed; the versions were not constrained, because that is a dependency move.
8. **No screen recording of the demonstration exists.** It is listed as missing rather than assumed, because it must be a recording of a real session or it does not exist.
9. **CI depends on an external host for a tokenizer**, so a red build with no defect behind it is possible.

18.4 Validity threats

These are the limitations that bear on whether the evidence in this report means what it appears to.

1. **A green local suite is not evidence about another environment**, and this project proved it twice — once with an integrity hash that had only ever passed locally, once with a full local sweep that passed five times and still failed remotely (§12.5, §12.6).
2. **The CI green carries three qualifications** (§12.6): a green run also occurred under the **old** budgets; the **per-scenario retry counts of the final green retry-enabled run are unknown, not zero**; and a green under two retries with a 180 s budget is **weaker evidence** than a first-attempt green under 60 s. Only the camera scenario's first-attempt 40.2 s remote pass, taken before any budget rise, stands unqualified.
3. **The raised CI budgets cost the suite a capability**. CI can no longer detect a genuine performance regression in that range. The budget is not described anywhere in this report as a fix; the stall remains real and unexplained.
4. **Byte-identical repeats are strong evidence of no state residue, not proof of none**.
5. **The residency experiment deliberately breaks the one-configuration-per-process rule**, so its figures are **not comparable** with the single-shot experiments — stated in its own record.
6. **The texture-fit decision rests on one reviewer, one decal, one camera**, and is scoped to the three production styles at the deck format rather than being a general claim about a 1.3× stretch (§14.4).
7. **No automated test loads the model**. A green suite is not evidence that the system generates anything (§15.2).
8. **The final matrix contained 24 duplicate generations** that were biasing a diversity indicator. Fixed in the plan and guarded by a test; the matrix was **not** regenerated, because its evidence is a valid superset of the fixed plan, and both fingerprints are recorded (§12.3).

18.5 What follows from these

None of these limitations invalidates the system or the research, and none of them is presented as fatal. Several of them are the most useful results the project produced: the reproducibility defect in §18.2.5 is a finding about how LoRA training pipelines silently fail to be deterministic, and the validity threats in §18.4 are a record of a project repeatedly discovering that its own verification was weaker than it looked.

The honest summary is that this system works, on this hardware, within a margin of about 2.4 %, and that four of its twelve research questions are only partially or boundedly answered — with RQ4's image-count component explicitly inconclusive. §19 draws conclusions on that basis and no wider one.

19 Conclusions

19.1 The primary question

How can a locally fine-tuned diffusion model, conditioned on both a text prompt and a reference image, generate skateboard-decal artwork in multiple visually distinct styles with reproducible quality on consumer hardware with 8 GB of VRAM?

It can, and this project establishes the specific configuration that fits.

Stable Diffusion 1.5 [1], generating directly at 512×1536, with one of three per-style LoRA adapters [3] at weight 0.7 and IP-Adapter [4] at scale 0.55, served by a single resident pipeline. Peak allocated 5143.73 MiB; **worst spare 200 MiB of 8187.5 MiB under real serving conditions.**

The answer is bounded in three ways that matter more than the affirmative:

- **The margin is 2.4 %.** The configuration fits; it does not fit comfortably, and anything added to it has 200 MiB to fit into.
- **"Reproducible" holds for inference and not for training.** A clean clone reproduced an output byte for byte three days later in a fresh environment. Training cannot be reproduced from its recorded seed, so the adapters are artifacts rather than recipes.
- **"Multiple visually distinct styles" is three, and one of them is a partial pass.**

19.2 Conclusions per research question

RQ	conclusion	strength
RQ1	LoRA is demonstrated feasible on 8 GB: rank 8, 3 133 MiB at 512 ² , 300 steps in 91 s	bounded — "most effective" not established; four alternatives screened, never run
RQ2	SD 1.5 is feasible; SDXL is not , at the resolution where it is better	strong, on two candidates
RQ3	A 148-item dataset of public-domain, CC0 and self-created work is sufficient and legally documentable	strong
RQ4	Style-only captions beat verbatim captions; image count: no conclusion	split — captions strong, count inconclusive
RQ5	Separate per-style adapters , because each style needs a different checkpoint step	strong
RQ6	IP-Adapter over img2img , decided by near-copy behaviour rather than by quality	strong
RQ7	Reference strength and adapter weight dominate perceived control	partial — rank and LR not swept

RQ	conclusion	strength
RQ8	Generate the deck ratio directly. The hypothesis that this would degrade was refuted	strong
RQ9	UV layout alone controls orientation; textures swap at runtime	strong
RQ10	A pre-declared rubric with fixed seeds and blinding at the first gate gives usable comparability	adequate — one human approver, AI-assisted analysis, no second rater
RQ11	Licensing and privacy are settled; memorisation is not	partial
RQ12	A documented two-process local run, validated by a clean clone	strong

Eight of the twelve research questions are answered within their stated scope. Four are only partially or boundedly answered: RQ1, RQ4, RQ7 and RQ11. RQ4's image-count component remains explicitly inconclusive. §18.2 states exactly what limits each.

19.3 The findings worth carrying beyond this project

Four findings are useful beyond this assignment as engineering lessons, **without claiming statistical generalisability** — they rest on one GPU, one human approver and the validity threats in §18.4.

A successful run is not a run that fitted. SDXL reported 30 of 30 successes at 1024×1024 while allocating 10 738 MiB on an 8 187.5 MiB card, because the operating system spilled into host memory instead of raising an error. Reading memory against the ceiling rather than against the exit code is the single most useful habit this project adopted, and it was adopted because one experiment nearly concluded the opposite.

The cheapest method can be disqualified by what it does, not by what it costs. img2img adds zero VRAM — decisive on an 8 GB budget — and was rejected because at the deck format it returns its own input. Every near-copy flag in that milestone came from it.

Training longer made the model less obedient. Prompt adherence fell from 4 to 3 at step 600 for two of three styles while style consistency held at 5. Two of the three shipped checkpoints are therefore step 300. This is visible only if you checkpoint several times and let a human compare.

Determinism has halves. Inference here is byte-reproducible across machines and days. Training is not reproducible at all from its recorded seed, because the adapter initialisation draws from an unseeded global generator — a defect that produces perfectly valid-looking runs and is invisible unless you compare weights.

19.4 On the process

The assignment assesses the research process, and the honest conclusion about it is that **the process was worth more than the result three separate times.**

The prototype ladder caught a geometry mismatch that four milestones of convenient test assets had hidden. The clean-clone test caught an integrity control that had only ever passed on the machine that wrote it. The two-gate review protocol caught a checkpoint selection that a single global step count would have got wrong for two styles out of three.

None of those would have surfaced from building the system and documenting it afterwards. Each came from a deliberately awkward step — build the next prototype rather than assume, clone into an empty directory rather than trust, stop and let a human look rather than let an indicator decide.

19.5 What this project does not conclude

It does not conclude that LoRA is the best fine-tuning method, that SD 1.5 is the best base model, that three styles is enough, that 40 images per style is a threshold, that the system is unbiased, or that a green test suite means the generator works.

Each of those would have been an easy sentence to write and none of them is supported by what was measured.

20 Reflection

20.1 What changed from the original plan

The plan was written on day one and never rewritten; thirteen change-log entries record what actually happened instead.

Almost everything finished early, and the buffer compounded rather than being spent. The viewer and the dataset both landed on the first day, which let the benchmark start three days early, which let conditioning start three days early, and so on to a feature freeze brought forward by six days. Only the testing milestone overran, by about an hour, and it is the one that found five real defects.

The under-runs were not luck, and two of them changed later estimates. Training turned out to cost 91 seconds for 300 steps, so the long training run the schedule had budgeted for never existed. The infrastructure built for early prototypes was reusable to a degree the estimates had not anticipated — the MVP added no modelling of its own, which is also why its memory behaviour is directly comparable with experiments from three milestones earlier.

Two features were added that were not in the plan, both scoped after using the application rather than while designing it: real generation progress, and local decal upload. Neither was scope creep in the harmful sense — both are logged with reasons — and the progress work produced a decision record and found three defects no test could see.

20.2 What failed

A style did not learn what it was supposed to. `retro-poster` learned the frames and typography of its source posters. The uncomfortable part is that this was **predicted before training** — a border-darkness measurement flagged 35 of 36 items — and the mitigation chosen, captions rather than cropping, was tested on a different style. It ships as a partial pass because that is what it is.

A research question did not resolve. The image-count arms came out non-monotonic. There was a temptation to present the ordering as a trend; the result is recorded as inconclusive instead, and the confound that makes it hard to interpret was declared in code before any result existed.

The reproducibility of training was broken the whole time and nobody noticed for four milestones. Every run passed every gate. Losses fell. Adapters loaded and changed output. Nothing looked wrong, because nothing was wrong except the thing nobody was checking. It was found only when two runs of the same configuration were compared at the weight level.

An integrity check had never actually run. The dataset hash passed on every machine it was ever tested on — which was one machine. A clean clone failed it immediately.

20.3 Decisions that saved the project time

Procedural deck geometry removed model sourcing, licensing and UV repair from the first milestone.

Micro-gating before long runs — 1 step, then 10, then 300, each authorising the next from a measured projection — felt slow and meant no long run ever started from a guess.

Stopping at human gates with nothing concluded. Twice the milestone finished its measurements and stopped, with a draft decision record containing no decision. It is the least efficient-looking practice in the project and it is what makes the conclusions defensible.

Building both options instead of arguing for one. Both texture-fit modes were implemented, each disclosing its cost numerically, with a test asserting no default was exported so a human had to choose.

20.4 Evidence that changed a decision

Four times the measurement overruled the plan:

- direct tall generation was expected to degrade and was **better** on every axis measured;
- SDXL was expected to be marginal and turned out to be **impossible** at the resolution where it wins;
- img2img was expected to be a reasonable low-VRAM default and turned out to **copy its input**;
- a hypothesis about thermal throttling was **tested and refuted** by a hotter card running faster.

The habit underneath all four is that each expectation was written down first, so it could be seen to be wrong.

20.5 Lessons that were expensive

Check the artefact, not the description of it. A style was labelled `retro-comic` for days; the material is silkscreen posters with no comic properties at all. The wrong label had already reached the prompt kit. It was caught by opening the images. The same lesson recurred when a reference image described as "the framed one" turned out to be one of two.

A test that has only ever passed in one place has not passed. This applies to the dataset hash, to the local test sweep that passed five times and failed remotely, and — uncomfortably — to any claim in this report that rests on a single environment.

Instrument the measurement before trusting it. A 20× timing spread had an obvious explanation that was wrong, and the real cause was the measurement design.

20.6 On working with an AI assistant

The assistant wrote most of the code and most of the documentation. Three things made that defensible rather than dangerous, and they are worth stating because none of them is automatic.

The gates were real. Visual analysis at the style-learning gates was AI-assisted — ChatGPT proposed scoring of the contact sheets — and **every recorded score was reviewed and approved by Kylian, who made every production selection and research conclusion himself.** Twice the milestone stopped with the work finished and nothing decided.

Plan review caught real errors before execution. The claim that gradient accumulation would save memory was wrong, and was corrected by the student before any GPU time was spent on it. A review gate that had been promised and then bypassed in the same plan was restored. These are not cosmetic corrections; the first would have produced a measurement of a thing that does not happen.

The assistant's own output was wrong often enough to matter. A runner defect, an ambiguous test selector, a fixture reader that could not parse real code, a reporting bug that presented missing data as a passed check. The last one is the instructive one: it failed in the project's favour, which is the direction errors are least likely to be noticed.

What did not work well: the assistant is comfortable producing confident prose about work it has just done, and that is exactly where a reader should be most careful. The counter-measure adopted — requiring every quantitative claim in this report to resolve against an evidence file at build time — exists because judgement alone was not a sufficient check.

20.7 What would be done differently

Seed everything on day one. The reproducibility defect cost the project the ability to regenerate its own production artifacts, and the fix is three lines. It was not written because determinism was assumed rather than tested.

Run the clean-clone test at the first milestone, not the eighth. It found five defects in an afternoon, every one of which had existed for weeks.

Design the image-count experiment for equal epochs, or state up front that it cannot answer the question asked. Matching on compute made the comparison affordable and made the result nearly uninterpretable.

Measure one alternative fine-tuning method properly, even a small one. DR-009's honesty about claiming feasibility rather than superiority is correct, and it is also a consequence of a budget decision that could have gone differently.

Start the report earlier. It was planned to run in parallel from the sixth milestone and did not. Nothing was lost because the process was documented as it happened — which is the only reason writing it at the end was possible at all.

21 Lessons learned

Each lesson below is tied to the moment that produced it. They are ordered by how much they changed the project's behaviour afterwards.

21.1 Technical

Read memory against the ceiling, not against the exit code. A model reported thirty successful runs while allocating 31 % more memory than the card physically holds. No exception was raised and no escalation triggered, because the operating system spilled into host memory. Every memory figure in this report is quoted against 8187.5 MiB as a direct result (§9.1).

Separate memory peaks by phase. Recording post-load, forward/backward and optimizer-step peaks separately, rather than one process maximum, is what revealed that activations scale with geometry while optimizer state does not — and therefore that gradient checkpointing was the correct first escalation and a lower-memory optimizer would have been the wrong move (§9.2).

One configuration per process. A shared process gave a 20× timing spread for identical work through allocator state, and the first explanation offered — thermal throttling — was wrong (§12.2).

A hoped-for result is a diagnostic, not a pass condition. An adapter at weight 0.0 producing byte-identical output is welcome evidence and must not be allowed to fail the experiment, because loading an inactive adapter can legitimately change the execution graph. Declaring it a diagnostic before the run is what kept it honest (§9.2).

A differing hash is not evidence of a meaningful change. The weight-1.0 arm required a noise floor declared before any result was read, not merely a different image.

Verify against the live object, not the call that returned. Adapter attachment is confirmed by reading 128 LoRA modules and 16 attention processors back off the UNet, because a call that does not raise is not evidence that anything attached.

Gradient accumulation is not a memory tier. At micro-batch 1 it changes effective batch size, not peak memory. Caught in plan review before any GPU time was spent, and now enforced by a guard.

21.2 Methodological

Write the threshold down before reading the result. Noise floors, tolerances and pass conditions all predate the runs they judge. This is the single practice that most distinguishes the experiments in this report from a set of demonstrations.

A hypothesis that equality satisfies cannot be refuted. One style-learning hypothesis read "at least as strong as" and was rewritten into four explicit verdict rules before it could be tested.

Blind first, label second, and hash the scores before unblinding. The first review gate's score file was hashed before the blinding map was opened, so "no score was edited afterwards" is checkable. The second gate had to be labelled — the question required it — and the expectation effect that introduces is recorded rather than left implicit (§6.3).

A blank is not a zero. Twenty-nine unscored cells were excluded from every mean rather than imputed, and one comparison was left unscored rather than reusing a plausible number from an earlier milestone.

Let automated indicators inform and never decide. They populate no rubric cell and select no checkpoint, and they live in separate files from human judgement.

Build both options and decide nothing. Used at three gates. The version of this that works has a test asserting no default was exported, so the choice cannot be made by omission.

21.3 Reproducibility

Determinism is not inherited; it is per-source-of-randomness. Seeding a generator for latents, noise and timesteps produced a fully deterministic data pipeline and a completely non-reproducible adapter, because one initialisation drew from the global generator. Everything looked fine for four milestones (§12.4).

Pin immutable revisions, and verify availability at the moment of use. Four separate third-party sources became unavailable during this project, one of them while the bibliography was being written (§24.4).

Some artifacts must be preserved rather than regenerated, and a system that depends on them needs a manifest, a restore path and runtime verification — not a note in a README (§16.3).

Line endings can break an integrity check. A hash-locked evidence file needed an explicit attribute to survive checkout on a machine configured differently from its author's.

21.4 Testing and CI

A green local suite is not evidence about a different environment, and this project proved it twice in one milestone (§12.5, §12.6).

Retries cannot help a stall that outlasts a retry cycle. Three attempts inside one stall window all failed while another test's retry, landing outside it, passed. Raising the retry count would have been the obvious wrong move.

Wait in frames, not milliseconds, when the thing you are waiting for advances per rendered frame. A camera test that polled on wall-clock time was making a bet on the frame rate, and lost it on a GPU-less runner.

A mocked suite must be pinned to the real contract. Frozen fixtures are validated against the real API models by a separate test, so a field rename breaks a test rather than leaving the browser suite passing against a shape that no longer exists.

A test count is only evidence if every test earns its place. Three planned tests were dropped after checking disproved their premise (§15.7).

21.5 Working with AI assistance

Gates are the mechanism, not good intentions. The boundary held because milestones stopped with work finished and nothing concluded, not because anyone remembered to be careful.

Review the plan, not just the output. The most valuable corrections in this project were made before execution: a memory claim that was false, a review gate that had been promised and bypassed in the same document, and an experiment design that would have answered a different question than the one asked.

Errors in your favour are the dangerous ones. A reporting bug that presented missing data as a passed check is harder to notice than one that breaks a build.

Make the claims checkable rather than trusting the prose. Every quantitative value in this report is resolved against its evidence file at build time, and a value that has drifted fails the build. That mechanism exists because careful writing was not a sufficient control.

22 What should be done differently

Six changes, ordered by what each would have bought. Each names the moment it comes from, so it can be judged against what actually happened rather than as general advice.

22.1 Seed every source of randomness on the first day

Cost of not doing it: the three production adapters cannot be regenerated. They are artifacts identified by SHA-256, and if they are lost the production selection is lost with them. Everything downstream — a manifest, a restore script, runtime verification on every style activation, and a whole risk entry — exists to manage a consequence that three lines of initialisation code would have prevented.

Why it was missed: determinism was assumed because a generator *was* seeded, for latents, noise and timesteps. Nobody asked whether anything else drew from a different source. It looked correct for four milestones because every run passed every gate.

Differently: treat "is this deterministic" as a test with an assertion, run at the first training run, comparing two runs of identical configuration at the weight level. It takes one afternoon.

22.2 Run the clean-clone test at the first milestone

Cost of not doing it: five real defects sat in the repository for weeks, including an integrity control that had never once been executed anywhere but its author's machine, and a settings file documenting five variables nothing reads.

Why it was missed: the clean clone was scheduled as a deployment-validation activity, which is where it conventionally belongs. It is actually a *correctness* activity, and it is cheap.

Differently: clone into an empty directory and run the suite at the end of the first milestone that produces a suite, and repeat it at every milestone boundary. Every defect it found was environmental, and every one was invisible locally by construction.

22.3 Design the image-count experiment to answer the question asked

Cost of not doing it: RQ4's count half is inconclusive, and it is the project's only genuinely unresolved research question.

Why it happened: matching the arms on equal *compute* made three training runs affordable inside the budget. It also meant the 12-image arm saw each image 25 times and the 44-image arm saw each 6.8 times, so set size and repetition moved together and the ordering came out non-monotonic.

The confound was **declared in code before any result existed**, which is why the result is honest rather than misleading — but honest and uninterpretable is still uninterpretable.

Differently: match on epochs and accept the extra GPU time, or state at design time that the experiment measures set size at fixed compute and is not capable of establishing a minimum count. The second is cheaper and would have set expectations correctly from the start.

22.4 Measure at least one alternative fine-tuning method

Cost of not doing it: DR-009 can claim feasibility and not superiority, and §18.2 has to say so in the limitations. The assignment asked for five methods compared; four were compared on criteria and never run.

Why it happened: a deliberate budget decision. Measuring one method properly was judged more valuable than measuring three badly, and on a 19-day timeline with 8 GB that was probably right.

Differently: run a minimal Textual Inversion arm — it is the cheapest of the four in both memory and time — on the lead style only, with the same rubric. Even a single measured comparison would move the claim from "feasible" to "feasible and better than one measured alternative", which is a materially stronger research position for a small cost.

22.5 Write the report in parallel, as planned

Cost of not doing it: the report was planned to run alongside the later milestones and did not start until they were finished, which concentrated the work at the end of the schedule.

Why it survived: because the process was documented as it happened. Every decision record, experiment row and process-log entry was written at the time, so the report is a synthesis of existing evidence rather than a reconstruction. **That is the only reason writing it at the end was possible at all** — and it is worth noting that the practice which saved it is the one the assignment was assessing.

Differently: draft each chapter at the close of the milestone that produces its evidence, while the reasoning is still available and before the numbers need looking up.

22.6 Establish the deck geometry from the real target immediately

Cost of not doing it: the mismatch between a 1:3 generated decal and a 1:3.902 UV domain was found in the fifth prototype, four milestones after the geometry was chosen, because the test decals happened to be 512×2000 — close enough to hide it.

Why it happened: the test assets were authored before there was anything to generate, so they were made at a convenient ratio rather than the ratio the pipeline would eventually produce.

Differently: make every placeholder asset match the exact dimensions the real pipeline will produce, or make it obviously wrong. A placeholder that is nearly right is the worst of the three options, because it postpones the discovery without preventing it.

22.7 What would not be changed

Three practices cost time, looked inefficient, and would be repeated without hesitation.

Micro-gating before every long run. No long run ever started from a guess.

Stopping at human gates with the work finished and nothing concluded. Twice a milestone ended with a draft decision record containing no decision. It is the reason the conclusions in §19 are defensible.

Preserving failed rows, refuted hypotheses and blocked experiments in the record. They are the most informative part of this report, and every one of them was available to be quietly deleted.

23 Future work

Ordered by what each would resolve. Every item names the limitation it addresses, so none of them is a wish rather than a consequence.

23.1 Close the reproducibility defect

Resolves: §18.2.5 — training is not bit-reproducible from seed.

Seed the global generator alongside the explicit one, before adapter construction, with regression tests in both directions. The runner fix is already authorised and written; what remains is the expensive half — **re-running the affected comparisons**, because the fix is deliberately forward-only and the existing evidence predates it.

Until that happens, the three production adapters remain artifacts rather than recipes, and the whole restore-and-verify apparatus in §16.3 stays load-bearing.

23.2 Exercise the external backup

Resolves: §18.3.5 — the restore mechanism is validated, the external drive is not.

A restore from the external backup drive into a clean clone, with the same hash verification the working-repository restore received. This is an afternoon's work and it closes the project's single highest-impact residual risk: the loss of unregenerable artifacts.

23.3 Measure a second fine-tuning method

Resolves: §18.2.2 — LoRA is selected on feasibility, not superiority.

A minimal Textual Inversion [6] arm on the lead style, scored with the same rubric against the same fixed-seed grid. It is the cheapest of the unmeasured four, and it would move DR-009 from a feasibility claim to a measured comparison.

DreamBooth [5] and full fine-tuning would need more memory than this machine has, so they remain future work **conditional on hardware** rather than on time.

23.4 Answer the image-count question properly

Resolves: §18.2.1 — RQ4's count half is inconclusive.

Re-run the size arms **matched on epochs rather than compute**, across more than one style, with set sizes chosen to separate count from repetition. This is the one research question this project failed to answer, and the reason it failed is a design decision rather than a measurement problem (§22.3).

23.5 Test the mitigation that was never tested

Resolves: §18.2.9 and the retro-poster partial pass.

Two things were never measured. A **crop pass** removing frames at the pixel level, which was considered and rejected because it would have altered the dataset for one style only. And a **caption A/B on retro-poster itself** — the existing A/B ran on the lead style, so the caption strategy that was supposed to mitigate the poster problem was never tested against the poster problem.

Either could move that style from partial pass to pass, and neither can be claimed without running.

23.6 Scale beyond one generation at a time

Resolves: §18.3.1 and §18.3.3 — a 2.4 % margin and a single-worker service.

With a ≥ 16 GB GPU: a second resident pipeline, or a queue with a separate worker process, becomes possible. **Note what else that hardware would reopen** — SDXL [2] at its native resolution, which this project's own rubric scored as the visual-quality winner and rejected only on memory, and the multi-style adapter, which was viable and unselected.

This is the item that would most change the product, and it is entirely gated on hardware rather than on any finding in this report.

23.7 Address the deployment questions this project did not

Resolves: §17.6.

A content-moderation position, provenance marking of generated output, a bias evaluation of generated images rather than only of training data, and a licence position for commercial use given that the base model's card states research intent [10].

These are not postponed features. They are questions a commercial deployment must answer before launch, and this report deliberately does not answer them.

23.8 Product work that is genuinely optional

Listed last because none of it is required by any limitation: batch generation, a gallery of previous results, more styles, user accounts, and export presets for real print production. Each is straightforward and none of it would strengthen the research.

23.9 The one thing that should happen first

§23.2, exercising the external backup, is an afternoon and removes the risk of losing artifacts that cannot be recreated. Everything else in this section can wait; that one is a race against a disk failure.

24 References

24.1 What is cited, and what is not

This project's quantitative claims are supported by **repository evidence**, not by literature. Every memory figure, timing, hash, count and rubric score in this report cites a file in the repository, because that is stronger evidence for a measurement taken on this machine than any external source could be.

External references are therefore used for one purpose only: to identify the models, methods, libraries and licence regimes the project **used**, so a reader can find the primary description of each. No external source is cited as evidence for a measurement reported here.

Nothing below was cited without being retrieved and read, with two exceptions that are stated explicitly in §24.4 rather than concealed.

24.2 Models and methods

- [1] R. Rombach, A. Blattmann, D. Lorenz, P. Esser and B. Ommer, "High-Resolution Image Synthesis with Latent Diffusion Models", arXiv:2112.10752, 2021; CVPR 2022. <https://arxiv.org/abs/2112.10752> (accessed 2026-08-11).
- [2] D. Podell, Z. English, K. Lacey, A. Blattmann, T. Dockhorn, J. Müller, J. Penna and R. Rombach, "SDXL: Improving Latent Diffusion Models for High-Resolution Image Synthesis", arXiv:2307.01952, 2023. <https://arxiv.org/abs/2307.01952> (accessed 2026-08-11).
- [3] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang and W. Chen, "LoRA: Low-Rank Adaptation of Large Language Models", arXiv:2106.09685, 2021. <https://arxiv.org/abs/2106.09685> (accessed 2026-08-11).
- [4] H. Ye, J. Zhang, S. Liu, X. Han and W. Yang, "IP-Adapter: Text Compatible Image Prompt Adapter for Text-to-Image Diffusion Models", arXiv:2308.06721, 2023. <https://arxiv.org/abs/2308.06721> (accessed 2026-08-11).
- [5] N. Ruiz, Y. Li, V. Jampani, Y. Pritch, M. Rubinstein and K. Aberman, "DreamBooth: Fine Tuning Text-to-Image Diffusion Models for Subject-Driven Generation", arXiv:2208.12242, 2022; CVPR 2023. <https://arxiv.org/abs/2208.12242> (accessed 2026-08-11). **Screened, never run** — see §9.2.
- [6] R. Gal, Y. Alaluf, Y. Atzmon, O. Patashnik, A. H. Bermano, G. Chechik and D. Cohen-Or, "An Image is Worth One Word: Personalizing Text-to-Image Generation using Textual Inversion", arXiv:2208.01618, 2022. <https://arxiv.org/abs/2208.01618> (accessed 2026-08-11). **Screened, never run** — see §9.2.
- [7] L. Zhang, A. Rao and M. Agrawala, "Adding Conditional Control to Text-to-Image Diffusion Models", arXiv:2302.05543, 2023. <https://arxiv.org/abs/2302.05543> (accessed 2026-08-11). **Compared on criteria, never implemented** — see §11.2.

- [8] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal et al., "Learning Transferable Visual Models From Natural Language Supervision", arXiv:2103.00020, 2021.
<https://arxiv.org/abs/2103.00020> (accessed 2026-08-11). The encoder family used both by IP-Adapter conditioning and by this project's similarity indicator — which is why §13.5 states that indicator is descriptive *within* a method rather than neutral *between* methods.
- [9] C. Lu, Y. Zhou, F. Bao, J. Chen, C. Li and J. Zhu, "DPM-Solver++: Fast Solver for Guided Sampling of Diffusion Probabilistic Models", arXiv:2211.01095, 2022. <https://arxiv.org/abs/2211.01095> (accessed 2026-08-11). The sampler used for every generation in this project.

24.3 Model weights actually used

- [10] Stable Diffusion v1-5, repository [stable-diffusion-v1-5/stable-diffusion-v1-5](https://github.com/Stability-AI/stable-diffusion), licence CreativeML OpenRAIL-M. <https://huggingface.co/stable-diffusion-v1-5/stable-diffusion-v1-5> (accessed 2026-08-11). **Pinned in this project at revision 451f4fe1...** . The model card states the model "is intended for research purposes only"; §17 addresses this against the project's commercial framing.
- [11] h94/IP-Adapter, licence Apache-2.0, providing [ip-adapter_sd15](#) and [ip-adapter-plus_sd15](#) among others. <https://huggingface.co/h94/IP-Adapter> (accessed 2026-08-11). **Pinned in this project at revision 018e4027...** ; the base variant was selected and the Plus variant measured and not selected (§11.2).

Both are pinned to immutable commit hashes. That practice was adopted after three separate third-party sources became unavailable mid-project (§12.1), and it is the reason the models this report describes can still be identified exactly.

24.4 Dataset sources and licence regimes

- [12] The Metropolitan Museum of Art, *Image and Data Resources* / Open Access policy.
<https://www.metmuseum.org/policies/image-resources> Under this policy The Met makes images of artworks **it believes to be in the public domain** available under a Creative Commons Zero designation. Source of the 55 ukiyo-e items. **Retrieval blocked — see the note below.**
- [13] Library of Congress, *Rights and Access*, Posters: WPA Posters digital collection.
<https://www.loc.gov/collections/works-progress-administration-posters/about-this-collection/rights-and-access/> Related, for Federal Theatre Project material:
<https://www.loc.gov/collections/federal-theatre-project-1935-to-1939/about-this-collection/rights-and-access/> The Library does not own rights to material in its collections and does not grant or deny permission to publish; **assessing rights and any necessary third-party permissions remains the user's responsibility**, and this project treats it that way. Source of the 41 retro-poster items. **Retrieval blocked — see the note below.**
- [14] Creative Commons, "CC0 1.0 Universal" public-domain dedication.
<https://creativecommons.org/publicdomain/zero/1.0/> (accessed 2026-08-11).

RETRIEVAL LIMITATION ON REFERENCES 12 AND 13

Both pages were attempted repeatedly from the authoring environment across five official paths and could not be fetched: the museum returned **HTTP 429** and the library **HTTP 403** every time. **Neither page is quoted here**, and no search-engine extract has been presented as though the page itself had been read. The official URLs are given so a reader with an unblocked connection can verify them directly. The full attempt record is in `docs/evidence/M9/reference-retrieval.md`.

These pages were never this project's primary licensing evidence. That evidence is `data/manifests/dataset-v1.csv`, which records the source URL, author, licence, collection date and permitted use for **every one of the 148 items**, captured at the moment each was collected — when those servers were reachable. The manifest is hash-locked and a test asserts it has not changed.

This is the **fourth** time third-party hosting has obstructed this project, after two dataset sources and one base-model repository (§12.1). The mitigation adopted after those three — record provenance at the moment of use — is why this one is a footnote rather than a gap.

24.5 Libraries and tools

Cited because the report makes behavioural claims about them — for example that the image-to-image pipeline runs `int(steps × strength)` denoising steps (§13.4), and that a prompt-only request raises while an image adapter is resident (§14). Both were established by reading the library source, not by inference.

- [15] Hugging Face Diffusers. `https://github.com/huggingface/diffusers` — version 0.39.0 pinned in this project.
- [16] Hugging Face PEFT. `https://github.com/huggingface/peft` — version 0.20.0, pinned and installed only after a parsed resolver report proved it moved none of the protected packages (§12).
- [17] PyTorch. `https://pytorch.org` — 2.13.0+cu126.
- [18] FastAPI. `https://fastapi.tiangolo.com` — the API framework selected in §8.2.
- [19] Three.js and React Three Fiber. `https://threejs.org` — the 3D stack selected in §8.3.
- [20] Playwright. `https://playwright.dev` — the browser test runner used for the 38 end-to-end scenarios (§15).

References 15–20 are identifying citations for tools whose exact versions are recorded in the repository's own requirements and lock files, which are the authoritative record of what this project ran. Where a claim about a library's behaviour appears in this report, the evidence is the experiment that established it, cited in place.

25 Appendices

Appendix A — Experiment index

40 rows in `experiments/registry.csv`. Each carries its research question, hypothesis, dataset version, pinned model revision, full configuration, hardware readings, duration, output paths, evaluation, conclusion, next action and commit.

ID	proto	question	outcome
EXP-001	1	Does PyTorch see CUDA on the audited GPU?	PASS — bf16 available as a documented fallback
EXP-002	1	Is SD 1.5 feasible on 8 GB?	Selected — 2 675.38 MiB, 4.069 s
EXP-003	1	Is SD 2.1 base feasible?	BLOCKED — HTTP 401 gating, nothing generated
EXP-004	1	Is SDXL feasible at 512 and 1024?	Rejected — 10 738 MiB at 1024, silent host spill
EXP-005	1	How should the deck aspect ratio be produced?	Direct 1:3 selected ; hypothesis refuted
EXP-007	2	Does IP-Adapter attach to the UNet, and at what cost?	PASS — 16 of 32 processors replaced, +1 248.69 MiB
EXP-008	2	Is img2img influence controllable by strength?	Monotone, 6/6; zero extra VRAM
EXP-008b	2	Is the shared-process comparison valid for img2img?	PASS — +0.000 % against clean processes
EXP-009	2	Is IP-Adapter influence controllable by scale?	Monotone, 6/6; selected , default 0.55
EXP-009b	2	Is the shared-process comparison valid for IP-Adapter?	PASS — +0.000 %
EXP-010	2	Does scale 0.0 reproduce the text-only baseline?	12/12 byte-identical ; diagnostic, not a pass condition
EXP-011	2	Which wins when prompt and reference conflict?	Prompt authority falls as influence rises; frame and pseudo-text transfer confirmed
EXP-012	2	Does IP-Adapter-Plus do better?	Not selected — no decisive advantage, higher VRAM
EXP-013	2	Does conditioning survive the deck format?	All 6 near-copy flags are img2img here ; IP-Adapter memory-critical
EXP-014	2	What do offline indicators say?	Monotone 6/6 both methods; 6 flags, all img2img

ID	proto	question	outcome
EXP-016a/b	3	Does one LoRA step run, and is the loop stable?	PASS — micro-gates before any longer run
EXP-016	3	Does a LoRA train, save and reload?	PASS — 300 steps in 91 s
EXP-017a/b	3	Does training fit at the deck format?	PASS — feasibility probe only, 1 and 10 steps
EXP-018	3	Does the adapter reload and change output?	PASS — 4/4 beyond a pre-declared noise floor
EXP-019a	3	Do LoRA and IP-Adapter coexist?	PASS — first attempt preserved as a failed row (runner defect)
EXP-019b	3	R12: does the combined stack fit the deck format?	PASS by 202 MiB
EXP-020...022	4	Do the three styles train with style-only captions?	PASS — pilots
EXP-023	4	Do verbatim captions differ?	Blinded A/B; style-only selected at Gate 1
EXP-024n12/n24	4	What is the effect of set size at equal compute?	INCONCLUSIVE — non-monotonic
EXP-025	4	Which pilot checkpoint goes forward?	108/108 generations at the cap
EXP-026	4	Do the pilots reproduce training images?	0 of 108 flagged
EXP-027	4	Does the lead style reach a usable state?	PASS — step 300 shipped
EXP-028	4	Does ukiyo-e?	PASS — step 600 shipped
EXP-029	4	Does retro-poster, and does it bake in frames?	PARTIAL PASS — step 300 shipped ; H4 confirmed
EXP-030	4	One multi-style LoRA or separate adapters?	Viable, not selected
EXP-031	4	How do the approved checkpoints behave across the matrix?	252 of a 432 cap
EXP-032	4	Does the trained stack still fit?	202 MiB for all four candidates; spill signature absent
EXP-033	4	Do the approved checkpoints reproduce training images?	0 of 252 flagged; coarse indicator only
EXP-034	5	Does the stack survive as a long-lived service?	Worst spare 200 MiB ; growth 0.00 MiB

ID	proto	question	outcome
EXP-035	5	Can the placeholder influence a prompt-only result?	Byte-identical at scale 0.0 — proven inert

EXP-006 and EXP-015 are **not runs**: those identifiers name the human scoring directories.

Appendix B — Decision records

17 records in `docs/decisions/`.

DR	decision	notable for
DR-001	Monorepo	one history as process evidence
DR-002	FastAPI + Pydantic	validation weighted at 5 for untrusted uploads
DR-003	React + Vite + TS + R3F	validated by Prototype 0 rather than argued
DR-004	Diffusers + PEFT + Accelerate	kohya fallback retired only after measurement
DR-005	Procedural deck geometry	removed model licensing entirely
DR-006	Three styles and their sources	graffiti replaced by ukiyo-e on licensing risk
DR-007	SD 1.5, direct 1:3 at 512×1536	rests on two measured candidates
DR-008	Standard IP-Adapter @ 0.55	img2img retained as a labelled fallback
DR-009	LoRA, rank 8	claims feasibility, not superiority
DR-010	Three per-style adapters @ 0.7	two of three ship at step 300
DR-011	Single-process resident service	a memory measurement, not a preference
DR-012	<code>full-surface</code> texture fit	student's rationale quoted verbatim
DR-013	Real progress telemetry	no percentage without a denominator
DR-014	Native local deployment	Docker screened out, not benchmarked
DR-015	Report build pipeline	zero new dependencies under the freeze
DR-016	Presentation build pipeline	extends DR-015; the deck inherits the report's fact locks
DR-017	Deck length for a 20-minute slot	a constraint absent from the repository could not be checked

Appendix C — Research-question matrix

Eight of twelve answered within their stated scope: RQ2, RQ3, RQ5, RQ6, RQ8, RQ9, RQ10, RQ12. Four bounded or partially answered: RQ1, RQ4, RQ7, RQ11. RQ4's image-count component is explicitly **inconclusive**.

The full statement of each question is in §5.2, the status table in §26.3, and the conclusions drawn in §19.2.

Appendix D — Risk register summary

ID	risk	status
R1	8 GB insufficient for the chosen configuration	mitigating — quantified both sides
R3	Dataset licensing gaps	open — policy enforced pre-collection
R5	Windows CUDA/tooling friction	closed — occurred, resolved, pins recorded
R8	Style-learning quality plateau	open — one partial pass
R9	Evaluation subjectivity	open — AI-assisted analysis, one human approver, no second rater
R10	3D model licensing	closed — procedural geometry
R11	Third-party hosting unavailable	occurred four times (§12.1, §24.4)
R12	Combined stack does not fit	re-scoped, not closed — fits by 200 MiB
R13	Reference conditioning reproduces the reference	occurred , mitigated by method selection
R14	Training not bit-reproducible from seed	occurred — bounded, permanent
R15	Live demonstration fails on stage	mitigating — four-rung fallback
R16	Production checkpoints cannot be restored	mitigating — external backup not exercised

Appendix E — Fact locks

Every quantitative value repeated in this report is substituted at build time from `report/facts.yaml` and proved against the evidence file that states it. Extraction is graded: structural parsing for CSV and JSON, full 64-character matching for hashes, and context-anchored patterns for prose that must match exactly once — an ambiguous anchor is treated as a broken one, because it could confirm a superseded value as readily as the current one.

Sixteen tests assert this, six of them negative cases that prove the guards reject a wrong value, a stale value taken from the wrong column of the right table, an ambiguous anchor, a truncated hash, a missing source and a dead anchor.

Appendix F — Reproducing this document

```
python scripts/build_report.py      # sources -> HTML -> PDF
python scripts/validate_report.py   # structure, references, identifiers, facts
```

Chrome is a build prerequisite (DR-015). The PDF is reproducible in **content** from tracked sources; it is **not** byte-identical between builds, because Chrome embeds a creation timestamp and its own version. The recorded SHA-256 identifies the submitted artifact and makes no reproducibility claim beyond that.

26 D1–D7 traceability

The mapping below was appended to at every milestone in `docs/learning-outcome-traceability.md` rather than assembled at the end. Nothing in it is marked complete without a repository path behind it.

26.1 Learning outcomes

ID	outcome	evidence	report
D1	Independent applied research	12 research questions declared before any experiment · 40 registered experiments · results that refuted their own hypotheses (deck aspect ratio; thermal throttling) · one result left inconclusive rather than forced	\$5, \$12, \$13
D2	Independent professional functioning	a gated model escalated for a decision rather than authenticated around · a declared GPU cap reached exactly and respected · a new dependency admitted only after a parsed resolver report · two findings deliberately not fixed under a freeze, with reasons	\$12.1, \$7.3, \$18.3
D3	Iterative planning and methodology	planning v1 preserved verbatim + 13 change-log entries with reasons · a review gate split in two after plan review · a freeze brought forward to the day work finished	\$7
D4	Comparison of multiple solution methods	six weighted matrices · base model on measured data across two tracks · four conditioning arms · five fine-tuning methods weighed, one measured, with the limit stated · both texture-fit modes built rather than one argued for	\$8, \$9, \$11.2, \$14.4
D5	Complex problem solving via prototypes	six prototypes, none skipped, each answering the next one's precondition · a silent host-memory spill diagnosed · unseeded initialisation identified from the $\sqrt{2}$ shape of a discrepancy · five defects only a clean clone could find · a CI stall traced rather than retried at	\$11, \$12
D6	Justified research conclusions	every conclusion cites a registry row, a decision record or a commit · blanks never back-filled · visual evaluation AI-assisted, every recorded score reviewed and approved by Kylian, who held final authority · offline indicators populate no rubric cell · gate-1 scores hashed before unblinding, asserted by a test · the memory figure revised tighter on its own evidence	\$6.3, \$6.4, \$13, \$19
D7	Professional documentation	this report and its reproducible build · a runbook written for someone without this machine · a weights manifest guarded by a test · the application surfaces its own limitations to the user	\$16, \$14.6, DR-015

26.2 Mandatory requirements

#	requirement	evidence	status
1	Custom training dataset	<code>data/manifests/dataset-v1.csv</code> — 148 items	met

#	requirement	evidence	status
2	Provenance and permitted usage documented	per-item <code>source</code> , <code>licence</code> , <code>permitted_use</code> ; DR-006	met
3	Multiple visual styles (≥ 3)	three trained adapters; EXP-027/028/029	met , one a partial pass
4	Train or fine-tune locally	10 training runs on the audited GPU; DR-009	met
5	Text and reference-image conditioning	DR-008, IP-Adapter @ 0.55	met
6	Generate new decal artwork	31 real generations	met
7	Map onto a 3D skateboard	Prototype 0; DR-012	met
8	Reproducible deployment or demo setup	runbook + clean-clone test with real output	met
9	Public planning link	GitHub Project mirroring M0–M11	met
10	Research documentation as PDF	this document	met on delivery
11	Prototype evidence	<code>docs/evidence/</code> , six prototype documents	met
12	Final GitHub result	the repository	met
13	Presentation as PDF	<code>deliverables/DeckForge-AI-presentation.pdf</code> , DR-016	built, NOT yet gated

Requirement 13 was outstanding when this report was first built, and is now partially discharged. The presentation belongs to M10, which ran after the report's first build: **15 slides** are authored and both PDFs are tracked and produced by the pipeline of DR-016.

The authoritative slot — **20:00 including the live demo** — became known on 2026-08-14, after a 26-slide deck had already been built to a provisional length. It did not fit, by roughly 50 %, and **every structural check had passed on it**: a validator can only enforce a requirement it has been told. The deck was restructured to 15 slides against an enforced timing budget (DR-017), and estimated narration is now **14:56** against a **1:04** buffer.

It has still not passed a human visual gate, and no rehearsal has been run — every timing figure is an estimate from speaker-note word counts, not a measured delivery. "Built" is therefore not "met", and this row says so. No claim of full assignment completion is made anywhere in this report (§2.2).

26.3 Where each research question is answered

Eight of the twelve are answered within their stated scope. Four are only partially or boundedly answered: RQ1, RQ4, RQ7 and RQ11. RQ4's image-count component remains explicitly inconclusive.

RQ	answered in	status	verdict
RQ1	§9.2	bounded	feasibility established; " most effective " not established — four methods never measured
RQ2	§9.1	answered	on two measured candidates
RQ3	§10	answered	legally documentable dataset built
RQ4	§11.4	bounded	captions answered; image count INCONCLUSIVE
RQ5	§11.4	answered	per-style selected, multi-style viable
RQ6	§11.2	answered	IP-Adapter over img2img
RQ7	§11.2, §11.4	bounded	rank and learning rate not swept
RQ8	§9.1	answered	hypothesis refuted by its own result
RQ9	§11.0	answered	UV layout controls orientation
RQ10	§6.3	answered	with the subjectivity threat stated
RQ11	§17	bounded	licensing settled; memorisation not established
RQ12	§16	answered	validated by clean clone

26.4 What this matrix deliberately does not do

It does not mark a requirement met without a path, and it does not average partial results into whole ones. **Four research questions are marked bounded rather than answered, one component is marked inconclusive, and one requirement is marked not met.**

A traceability matrix whose every cell is green is not evidence that a project succeeded; it is evidence that the matrix was written to be green.